Construction, Validation, and Applications of High-fidelity Wireless Digital Twins



Yiming Li
yiming.li@duke.edu



Sam Rivera
sam.rivera@duke.edu



Tingjun Chen
tingjun.chen@duke.edu

Department of Electrical and Computer Engineering, Duke University

Overview of This Tutorial

Goal: Provide a practical, hands-on guide to developing ***scalable***, ***physically-accurate***, and ***high-fidelity*** wireless digital twins (DTs), with three main components:

1. **Geo2SigMap**: Scalable 3D scene generation using publicly available geographic databases, including the OpenStreetMap (OSM) building footprints and USGS's nationwide lidar datasets;
2. **NVIDIA Sionna**: An open-source library for research on communication systems, including differentiable ray tracing (RT) and electromagnetic (EM) compute engines;
3. **Closing the *sim2real* Gap**: Understanding and closing the gaps between real-world measurements the developed wireless DTs.

Overview of This Tutorial

Format: Lecture + hands-on labs (JupyterHub)

Outline:

1. **Overview and logistics**
2. **Part I**: 3D Scene Generation Using Geo2SigMap
3. **Part II**: Site-Specific Radio Map (RM) Generation and Ray Tracing (RT) Using Sionna RT
4. **Part III**: Connecting Wireless DTs with Real-World Measurements
5. **Summary and wrap-up**

Feel free to let us know if there are any questions throughout the tutorial!

Tutorial Setup

Instructions for setup can be found at: https://srivera2.github.io/milcom_wdt_tutorial_2026/

Option #1: Cloud-based setup (*recommended*): Follow instructions on the printout

Option #2: Local setup on your own machine:

1. From the sidebar ☰ select *"Local Setup"*
2. Ensure sure you have anaconda/miniconda installed
3. Walkthrough the instructions to create your environment
4. Before continuing, we suggest your run a cell to verify all libraries can be accessed

Warning: Generating 3D scenes may require high memory/storage overhead. If you are unsure about the capabilities of your machine, please consider using our remote server.

Please let us know if you are unfamiliar or need additional assistance!

Part I: 3D Scene Generation Using Geo2SigMap

Why Do We Need a 3D Scene?

A 3D scene is a digital representation of a real-world physical environment used to model radio propagation, sensing, and wireless system behavior

It provides the geometric, material, and semantic information needed for electromagnetic (EM) simulation and ray tracing (RT):

- **Geometry/Spatial Structure**: Buildings, terrain, vegetation and clutter, ...

  → *Represents the physical shapes and spatial relationships in the environment*

- **Material & EM Properties**: Reflection coeff., permittivity, conductivity, ...

  → Determines how radio waves interact with objects/surfaces

- **Radios**: Transmitter, receiver, ...

  → Each wireless node includes position/orientation, antenna configuration and pattern, ...

- **Dynamic Objects (*optional*)**: Moving vehicles, pedestrians, UAVs, ...

  → Enables modeling of mobility, Doppler, and time-varying scenarios

Representing a 3D Scene



Reuben Cooke Building

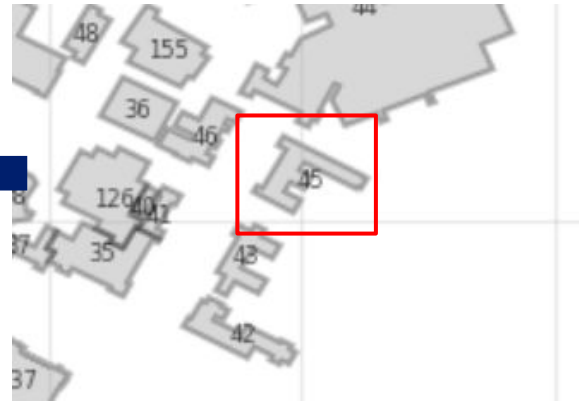
OSM Query



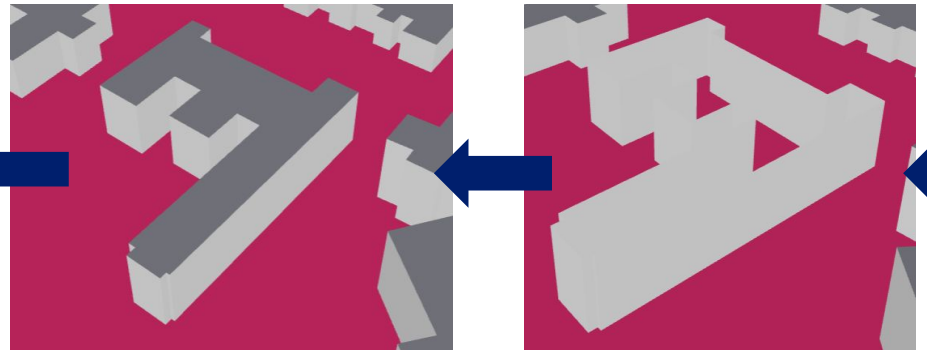
OSM .xml

```
<tag k="addr:city" v="Durham"/>
<tag k="addr:housenumber" v="417"/>
<tag k="addr:postcode" v="27705"/>
<tag k="addr:state" v="NC"/>
<tag k="addr:street" v="Chapel Drive"/>
<tag k="building" v="university"/>
<tag k="building:levels" v="4"/>
<tag k="name" v="Rueben-Cooke"/>
<tag k="source:addr" v="Durham Open Data"/>
```

Extract info



Convert to Local Coordinates

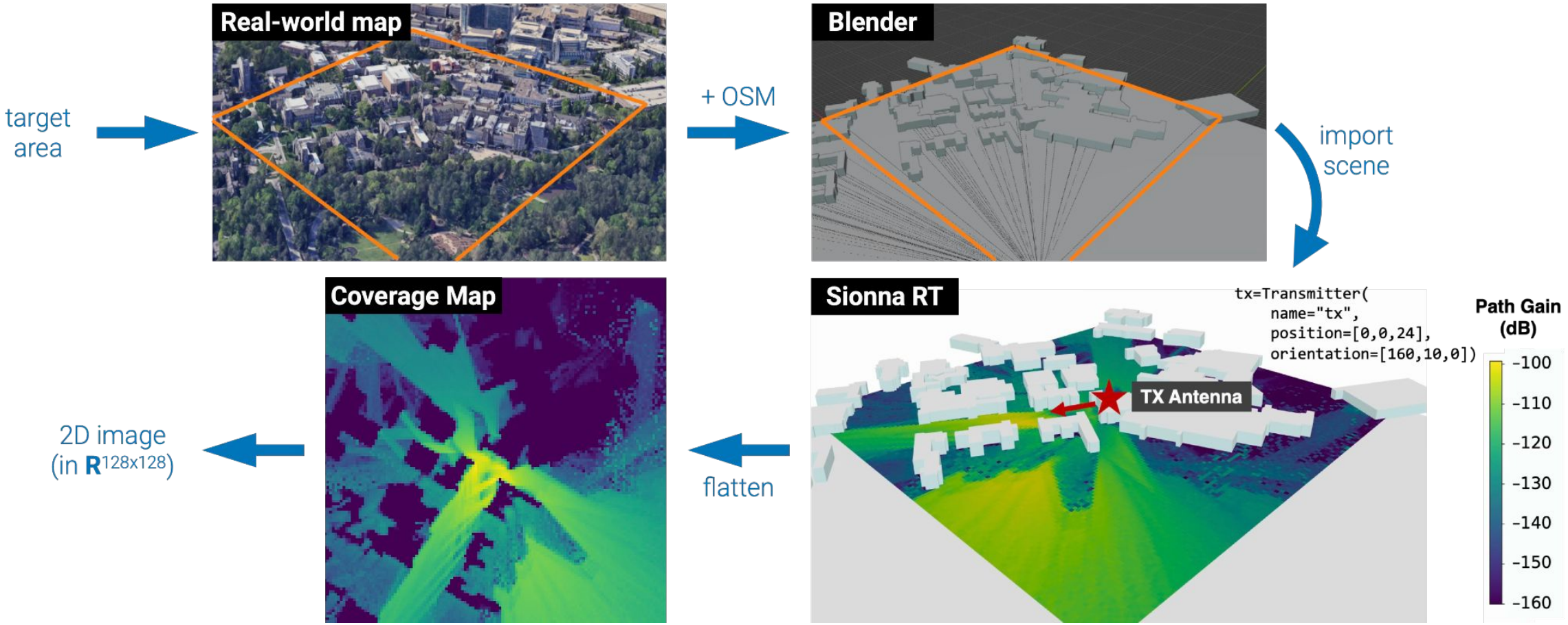


Create Triangle Mesh (.ply)
(Building Wall + Roof)

```
<shape type="ply"
id="mesh-building_45_rooftop">
    <string name="filename"
value="mesh/building_45_rooftop.ply"/>
    <ref id="mat-itu_metal" name="bsdf"/>
    <boolean name="face_normals"
value="true"/>
</shape>
<shape type="ply" id="mesh-building_45_wall">
    <string name="filename"
value="mesh/building_45_wall.ply"/>
    <ref id="mat-itu_concrete" name="bsdf"/>
    <boolean name="face_normals"
value="true"/>
</shape>
```

Mitsuba .xml

From 3D Scenes to Radio Coverage Maps



→ Blender is slow and not scalable
→ OSM info can be inaccurate or missing
→ The scene still lacks information (terrain, foliage, ...)

[1] Y. Li, Z. Li, Z. Gao, and T. Chen, "Geo2SigMap: High-Fidelity RF Signal Mapping Using Geographic Databases," in Proc. IEEE International Symposium on Dynamic Spectrum Access Networks (*DySPAN'24*), 2024.

3D Scene Generation Using Geo2SigMap

The first pure Python-based pipeline for 3D scene generation:

- Supports Sionna RT (.xml)
- Scalable, batched, ...

Supported Interfaces:

- CLI, Web UI, Python package

```
$ scenegen bbox -71.0602 42.3512 -71.0484 42.3591 --data-dir scenes/Boston

[INFO] Check the bbox at http://bboxfinder.com/#42.3512,-71.0602,42.3591,-71.0484
[INFO] Using UTM Zone: EPSG:32619
[INFO]
[INFO] Ground Material Type:           Wet Ground       | Frequency Range:   1   -  10   (GHz)
[INFO] Building Rooftop Material Type: Metal            | Frequency Range:   1   -  100  (GHz)
[INFO] Building Wall Material Type:    Concrete         | Frequency Range:   1   -  100  (GHz)
[INFO]
[INFO] Estimated ground polygon size: width=994m, height=901m
Parsing buildings: 100%|████████████████████| 389/389 [00:00<00:00, 1403.12it/s]
```

CLI (command line interface)



Web UI

Tools & Framework Design:

- OSMnx – OpenStreetMap (OSM) data query/extract
- Open3D – Triangle Mesh/3D Object Operations
- Triangle / SciPy (optional for BSD 3 license)
  – Delaunay Triangulation



OPEN3D

Triangle

Duke ENGINEERING FuNCtions Lab

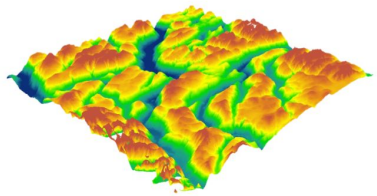
Geo2SigMap: https://github.com/functions-lab/geo2sigmap

IEEE MILCOM

3D Scene Generation Using Geo2SigMap



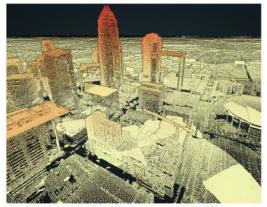
Target area
(*Duke U.*)



3DEP DEM Dataset



OpenStreetMap (OSM)



3DEP LiDAR Dataset

USGS science for a changing world

USGS science for a changing world

Calibration

Terrain

Buildings

Other Clutters

Mitsuba 3
Physically Based Renderer

(Mitsuba3 format)

USD
Universal Scene Description

(OpenUSD format)

Focus of this tutorial

NVIDIA Sionna

NVIDIA AODT

Mitsuba Renderer: https://www.mitsuba-renderer.org/; https://github.com/mitsuba-renderer/mitsuba3
Sionna RT: https://github.com/NVlabs/sionna-rt; https://nvlabs.github.io/sionna/rt/

README

Welcome to the Geo2SigMap. Our repository has been divided into two primary components:

- **Scene Generation**: A pure Python-based pipeline for generating 3D scenes for arbitrary areas of interest. This new Python-based pipeline replaces the scene generation pipeline used in our DySPAN'24 paper, and is more scalable, efficient and user-friendly.
- **ML-based Propagation Model**: ML-based signal coverage prediction using our pre-trained model based on the cascaded U-Net architecture, also described in our DySPAN'24 paper.

Our newest release **v2.0.0** has enhanced the original scene generation pipeline by incoporating LiDAR terrain data and downstream Digital Elevation Models (DEM). Data was sourced from the USGS 3D Elevation Program (https://www.usgs.gov/3d-elevation-program). The tool does not depend on this data for its operation. In areas with insufficient data, a flat terrain can still be assumed.

Overview

Geo2SigMap is an efficient framework for high-fidelity RF signal mapping leveraging geographic databases, ray tracing, and a novel cascaded U-Net model. Geo2SigMap features a scalable, automated pipeline that efficiently generates 3D building and path gain (PG) maps via the integration of a suite of open-source tools including OpenStreetMap (OSM) and Nvidia's Sionna Library. Geo2SigMap also features a cascaded U-Net model, which is pre-trained on pure synthetic datasets leveraging the building map and sparse signal strength (SS) map as input to predict the full SS map for the target (unseen) area. The performance of Geo2SigMap has been evaluated using large-scale field measurements collected using three types of user equipment (UE) across six LTE cells operating in the Citizens Broadband Radio Service (CBRS) band deployed on the Duke University West Campus. Our results show that Geo2SigMap achieves significantly improved root-mean-square error (RMSE) in terms of the SS map prediction accuracy compared to existing baseline methods based on channel models and ML.

Project Structure

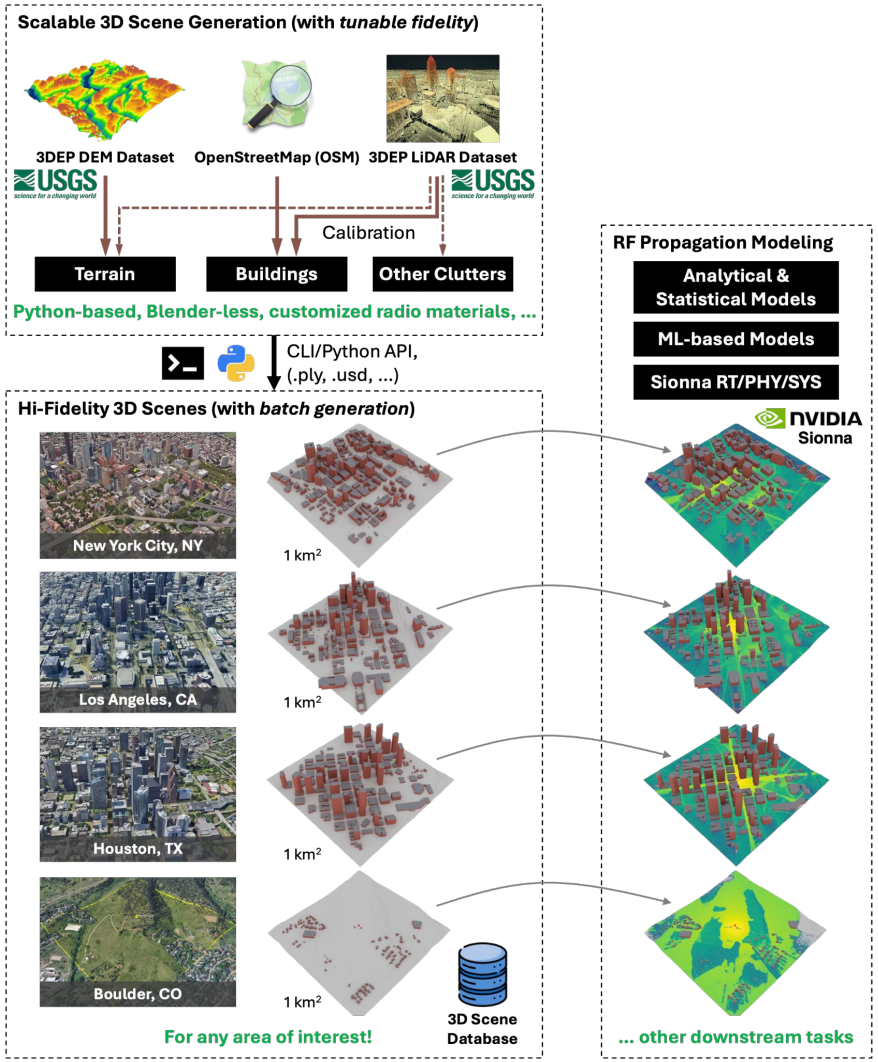
```
geo2sigmap/
├── package # Package Files
│   └── src
│       ├── scene_generation # Scene Generation Pipeline
│       ...
├── research # ML Model and Notebook Examples
│   ├── data # Datasets
│   │   └── measurements
│   └── examples # Example Notebooks
│       └── antenna_Pattern
...
```

GitHub: https://github.com/functions-lab/geo2sigmap

Geo2SigMap: Summary and Highlights

Release v2.0 features (Nov. 2025)

- Support of lidar datasets
- Support of irregular terrains
- Tunable scene fidelity
- Customized radio materials
- Batch scene generation
- ...

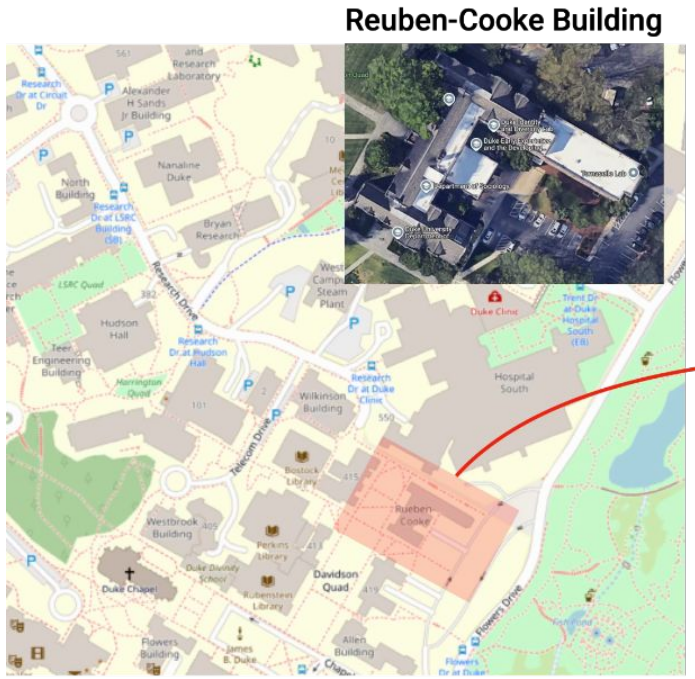


Geo2SigMap: https://github.com/functions-lab/geo2sigmap

OpenStreetMap (OSM)

A worldwide, publicly available database with 2D building shapes

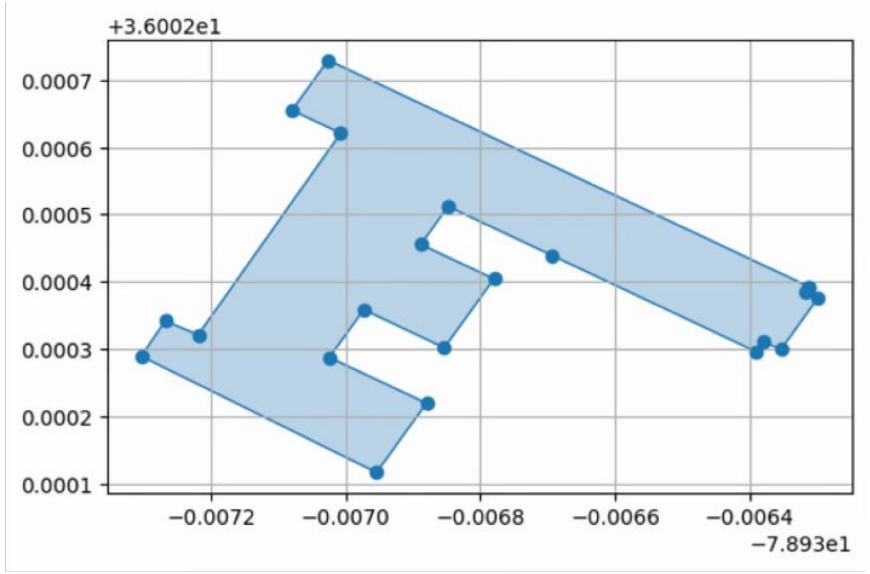
- Each building is represented by a **polygon** described by its vertices' GPS coordinates
- Building height information may not be accurate → can be corrected using lidar datasets



Example map on Duke campus

```xml
<?xml version="1.0" encoding="UTF-8"?>
<osm version="0.6">
    <bounds minlat="36.0006100"
        minlon="-78.9393000"
        maxlat="36.0039800"maxlon="-78.9335200"/>
    <way id="44164280" visible="true" version="7"
        changeset="131464781" timestamp="2023-01-
        19T12:31:08Z" user="fullExtent"
        uid="1035484">
     <nd ref="561034650"/>
     <nd ref="561034651"/>
     ...
     <nd ref="561034650"/>
     <tag k="addr:city" v="Durham"/>
     <tag k="addr:housenumber" v="417"/>
     <tag k="addr:postcode" v="27705"/>
     <tag k="addr:state" v="NC"/>
     <tag k="addr:street" v="Chapel Drive"/>
     <tag k="building" v="university"/>
     <tag k="building:levels" v="4"/>
     <tag k="name" v="Rueben-Cooke"/>
     <tag k="source:addr" v="Durham Open Data"/>
    </way>
...
</osm>
```

OSM .xml file

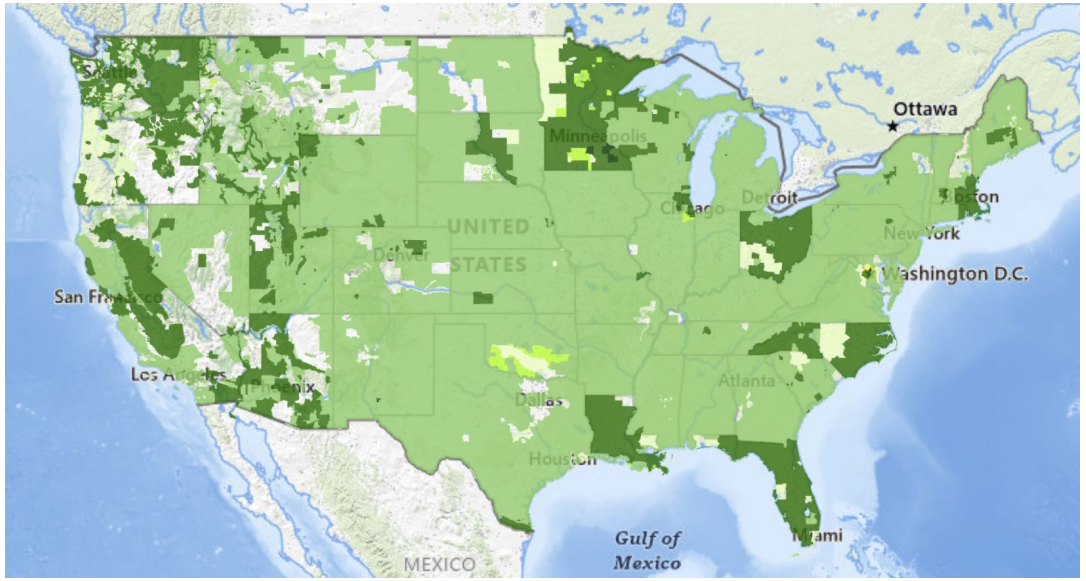


2D building polygon

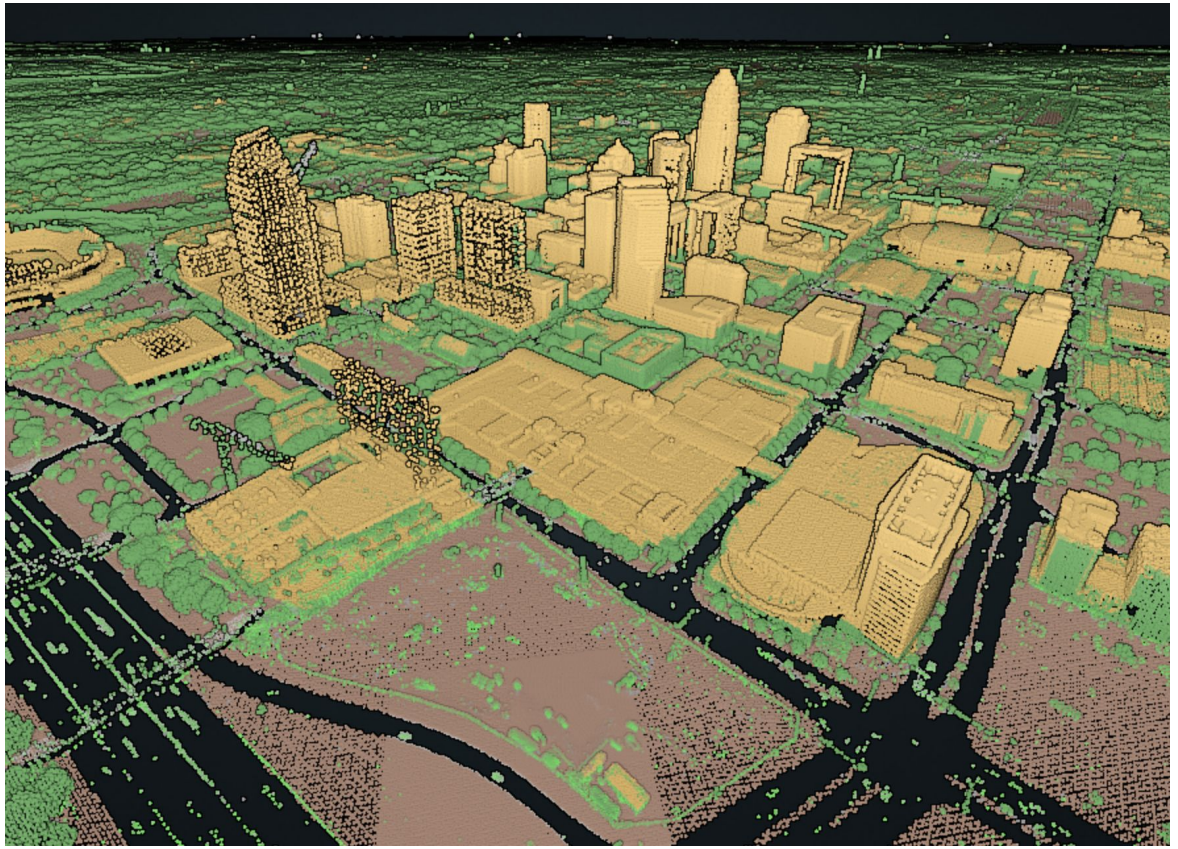
Feature #1: USGS 3DEP Lidar Dataset

Geo2SigMap leverages a nationwide publicly available lidar dataset for the continental U.S., provided by USGS. Key lidar point cloud properties include:

- Intensity
- (x, y, z) coordinates
- Classification (ground, building, foliage, ...)



USGS lidar data availability map



Example lidar point cloud (downtown Charlotte, NC)

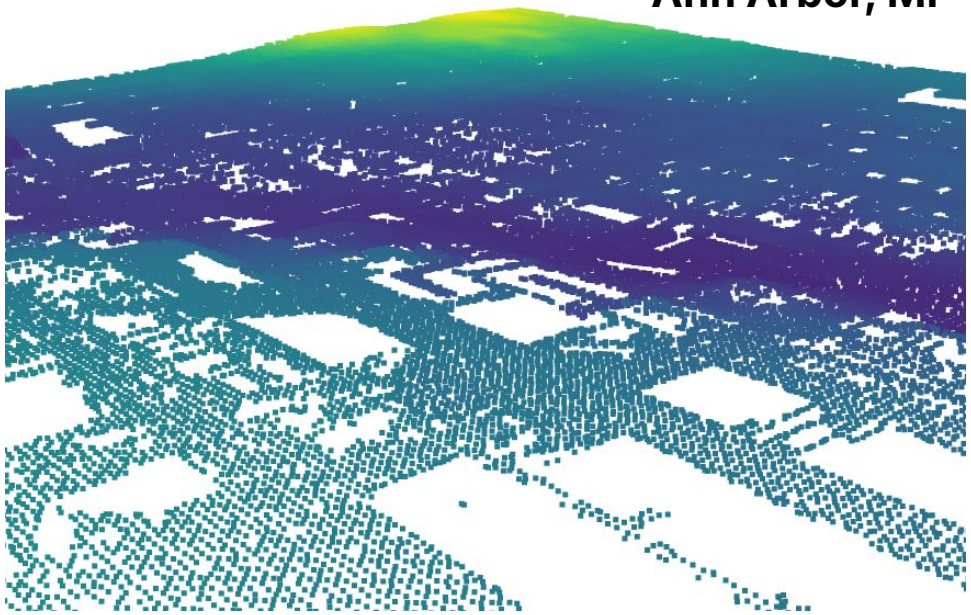
USGS 3D Elevation Program (3DEP): https://www.usgs.gov/3d-elevation-program

Feature #2: Lidar-based Irregular Terrains

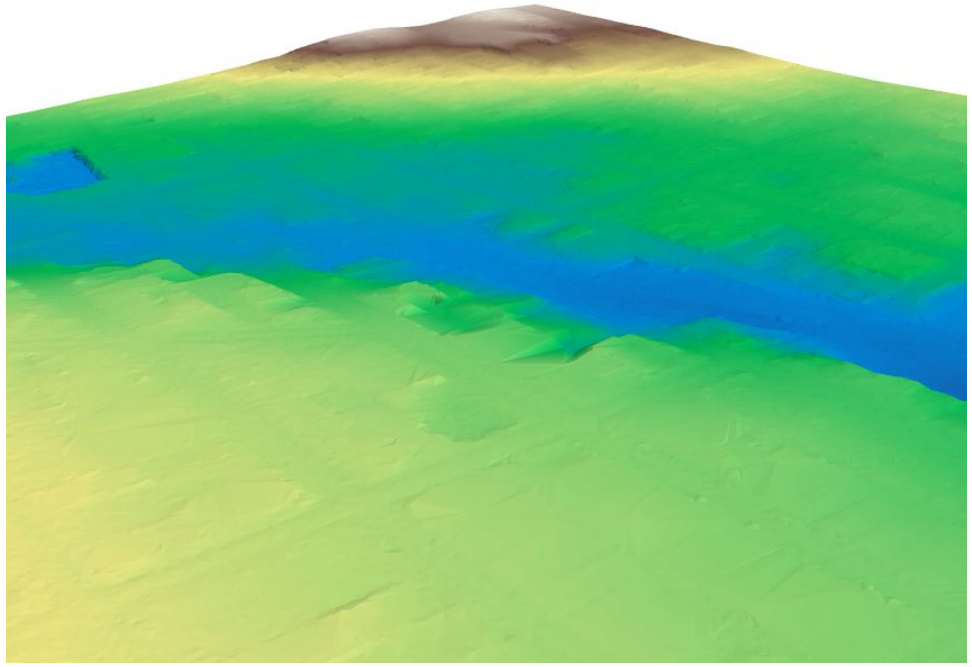
Direct terrain mesh generation from lidar point clouds with controllable detail

Method #1: PDAL + Laspy for preprocessing, filtering ground points; PyVista for mesh reconstruction (*all in Python*)

Ann Arbor, MI



Lidar point cloud (points labeled "ground")

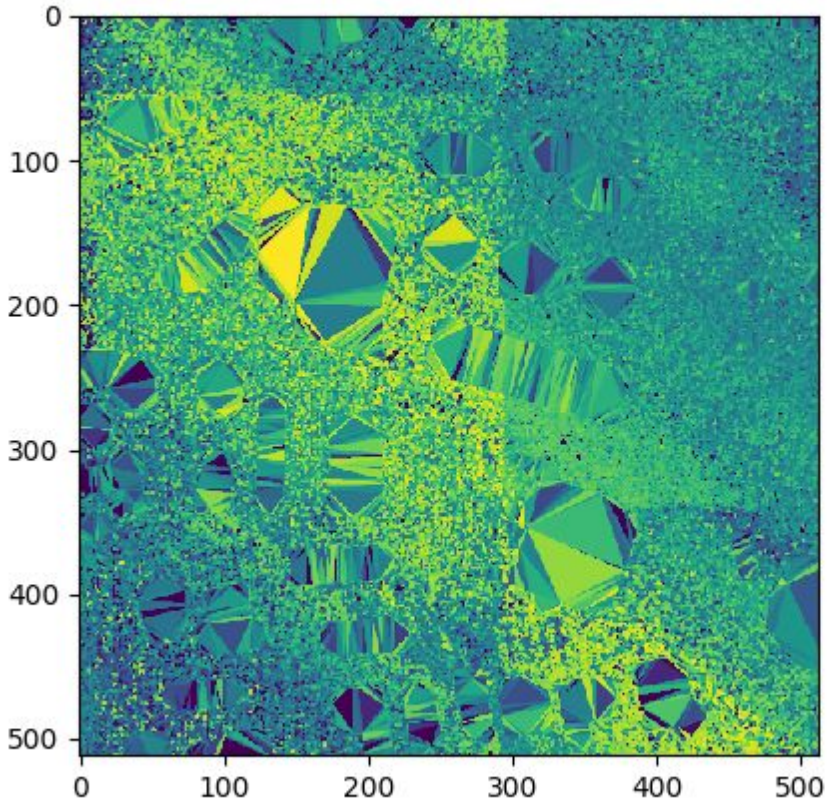


Constructed terrain mesh

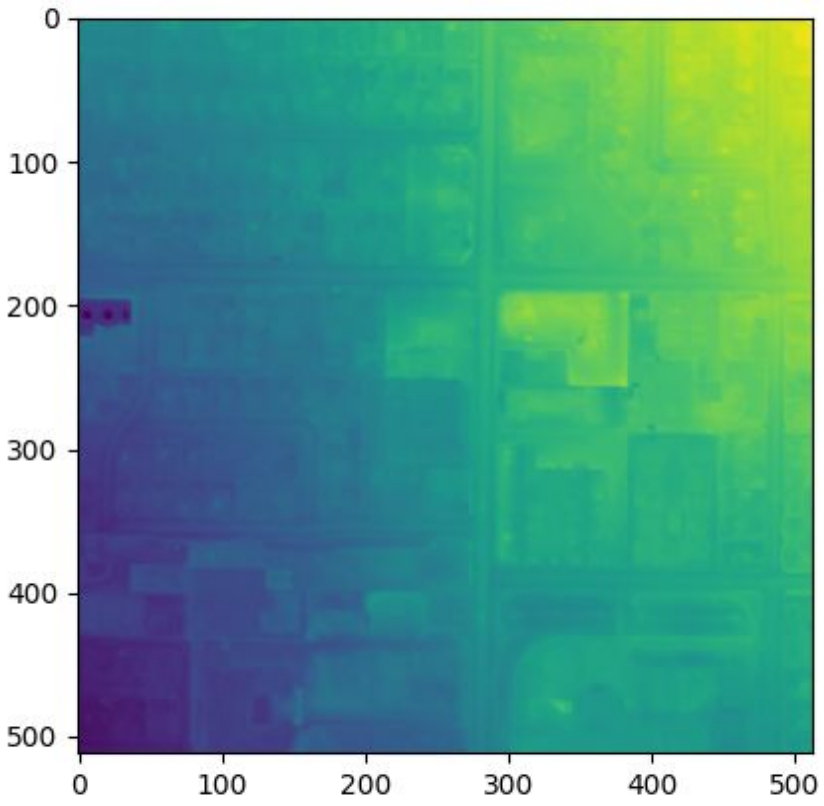
Feature #2: Lidar-based Irregular Terrains

Direct terrain mesh generation from lidar point clouds with controllable detail

Method #2: Using the USGS 3DEP digital elevation model (DEM) terrain mesh



“Skewed” terrain mesh



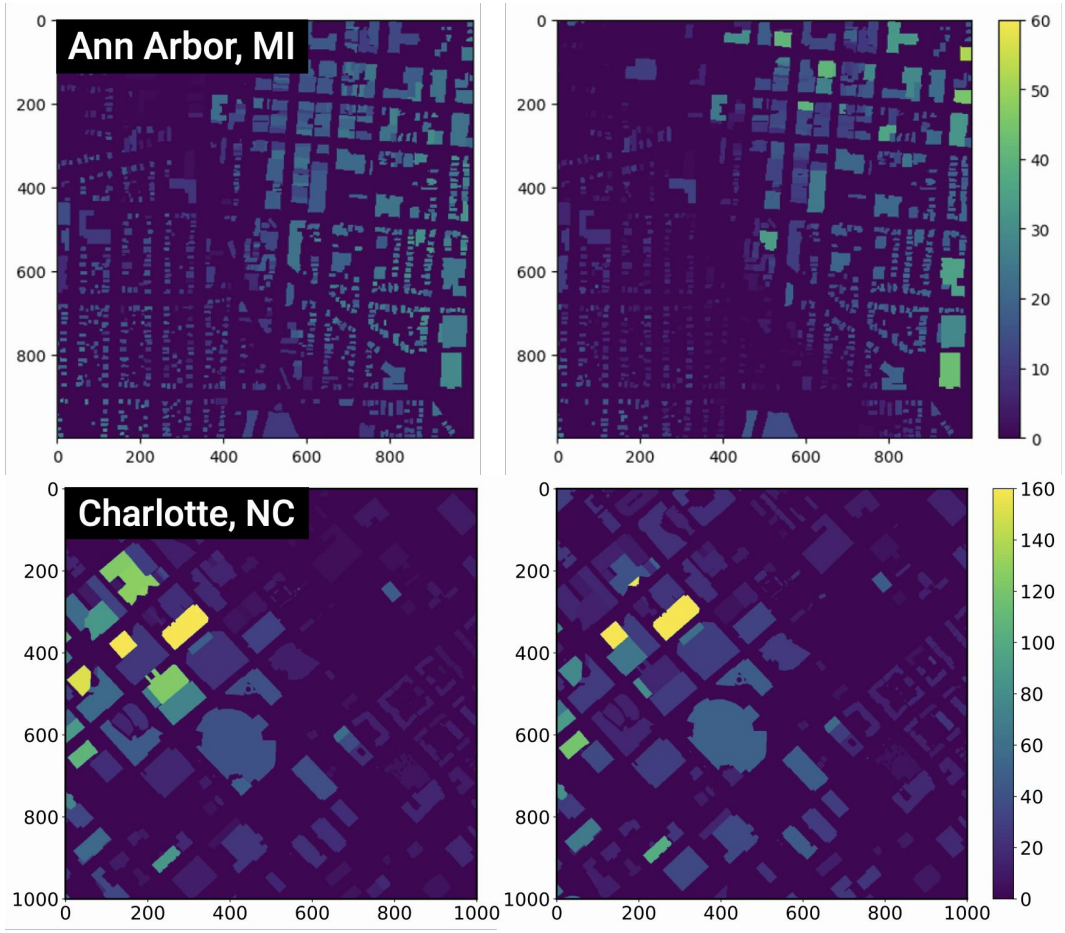
DEM-based terrain mesh

USGS 3D Elevation Program (3DEP): https://www.usgs.gov/3d-elevation-program

Feature #3: Building Height Calibration

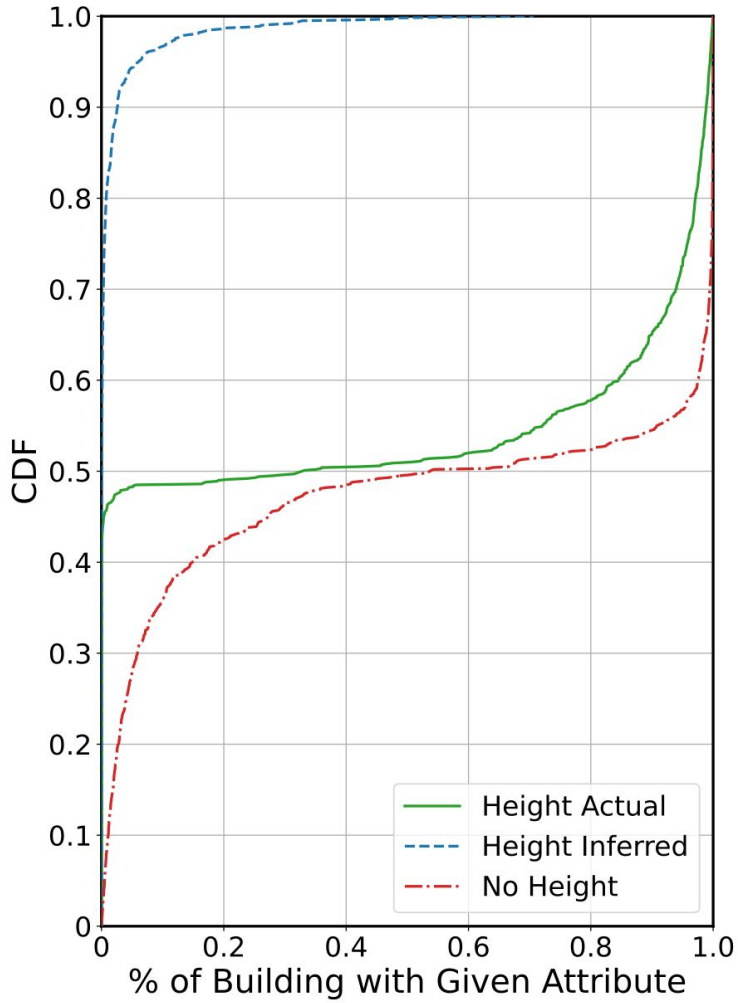
Building height can be inaccurate or missing in OSM!

Solution: Calibration based on lidar point clouds (z-values)



Building height map (raw OSM data)

Building height map (calibrated by lidar data)

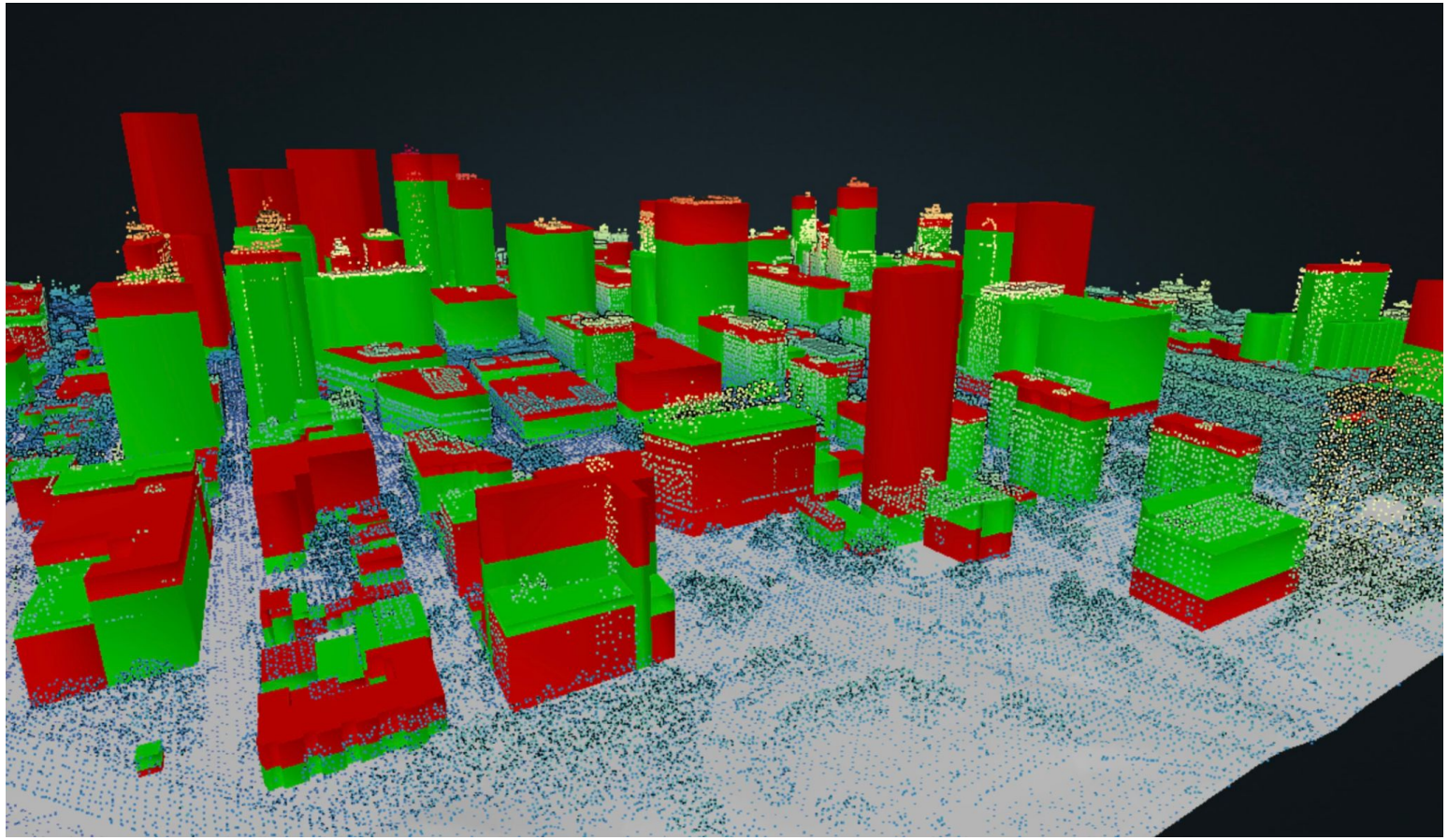


CDF of % of buildings with given attribute across 1,000 areas

Feature #3: Building Height Calibration

Building height can be inaccurate or missing in OSM!

Solution: Calibration based on lidar point clouds (z-values)

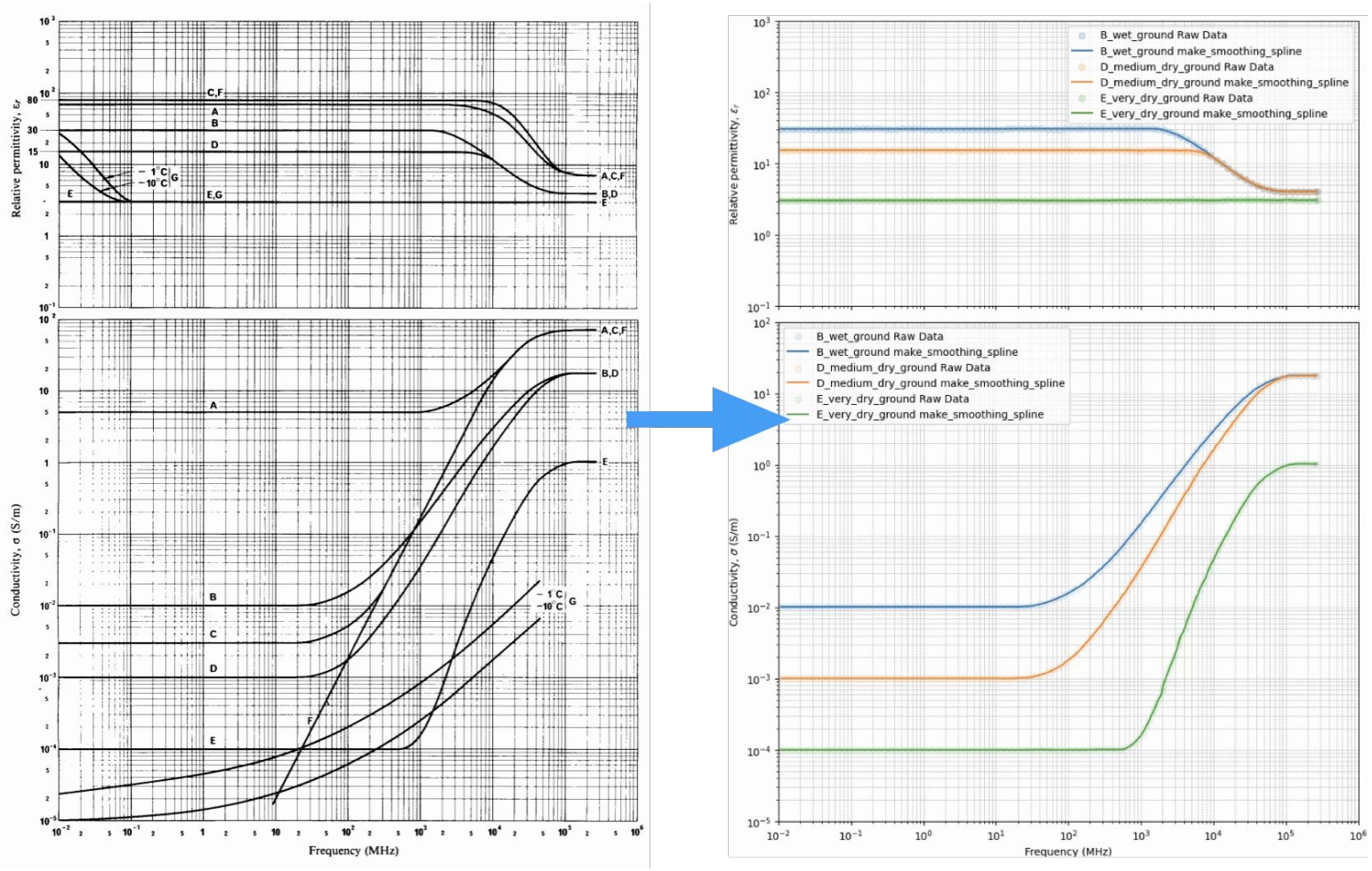


Visualizing the lidar-based building height calibration process

Feature #4: Customized Radio Materials

Current Sionna support: ITU-R P.2040-3 (***1–10 GHz***)

→ Extended range: ITU-R P.527-3 (***1 MHz–100 GHz***) using computer vision tools

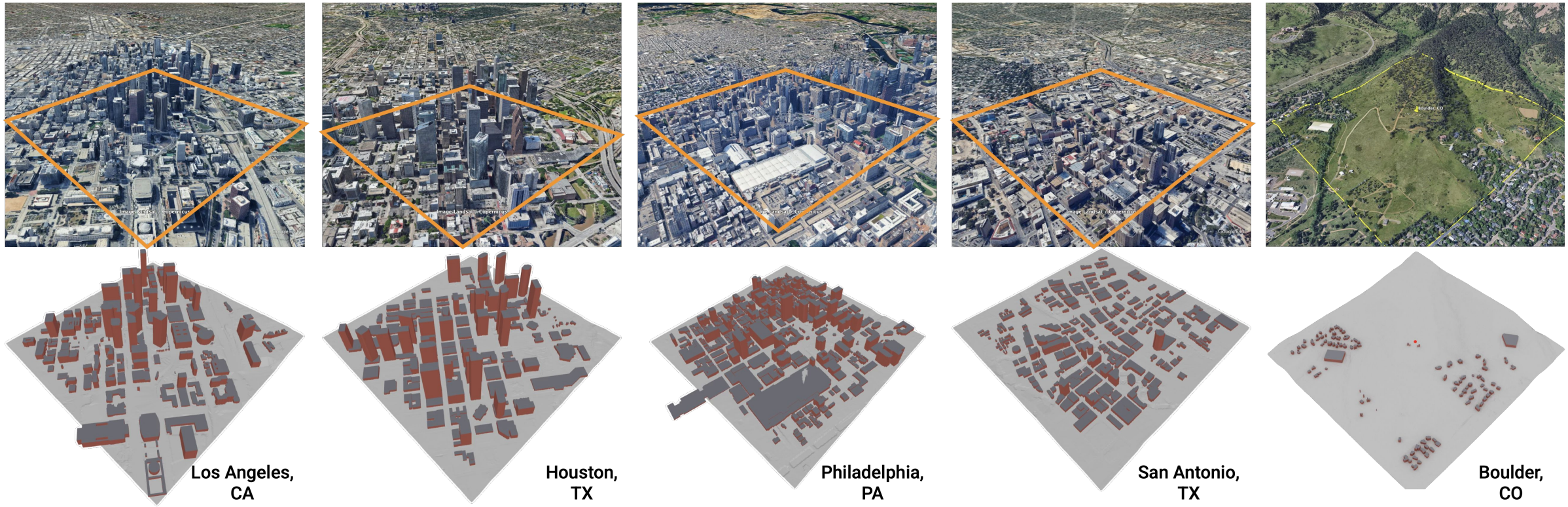


https://automeris.io/

Feature #5: Batched Scene Generation

Scalable, batched 3D scene generation across 10s of 1000s of areas

→ Allow efficient generation of large scene datasets (and their radio maps) for ML tasks



Geo2SigMap: https://github.com/functions-lab/geo2sigmap

Geo2SigMap API: Single Coordinate Method

```
# Replace '../scenes/Boston/scene.xml' with the path
to your scene file
SCENE_FOLDER = "../../scenes/Dupont_flat"

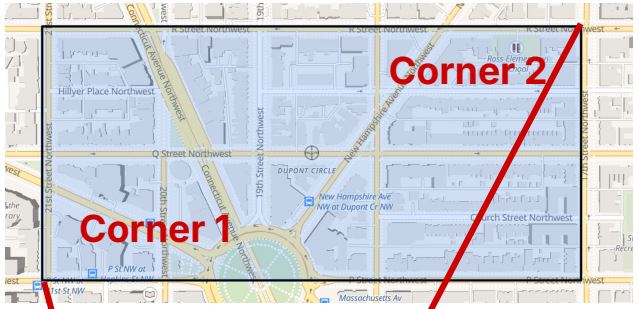
# Configure the width/height of the scene
scene_width = 256.0
scene_height = 256.0

# Picking an initial lat/lon
# Using the sta
points = rect_from_point_and_size(
   lon=-77.046,
   lat=38.909,
   position="bottom-left",
   width=scene_width,
   height=scene_height
)
```



```
# Generate scene if it doesn't already exist
if not os.path.exists(f"{SCENE_FOLDER}/scene.xml"):
   # Generating the scene using the Python API
   print("Generating a flat scene for Dupont Circle")
   scene_gen = Scene()
   scene_gen(
       points,
       SCENE_FOLDER,
       None,
       osm_server_addr="https://overpass-api.de/api/interpreter",
       lidar_calibration=False,
       lidar_terrain=False,
       dem_terrain=False,
       generate_building_map=True,
       ground_material_type="mat-itu_concrete",
       rooftop_material_type = "mat-itu_metal",
       wall_material_type = "mat-itu_brick"
   )
```

Geo2SigMap API: CLI + bbox





Box style

Command

Save dir.

```
$ scenegen bbox -77.046 38.909 -77.038 38.912 --data-dir scenes/Dupont_lidar --enable-lidar-terrain --enable-dem-terrain

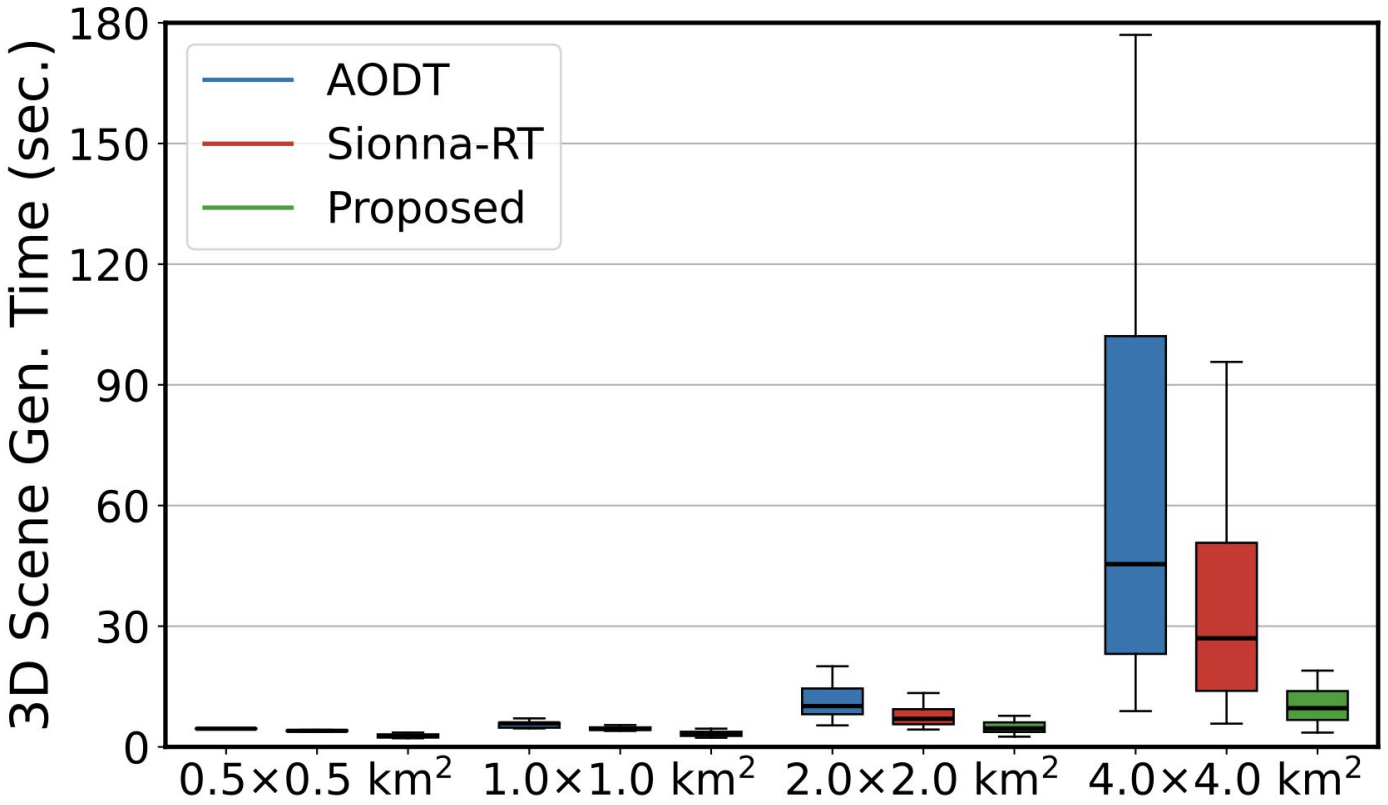
[INFO] Check the bbox at http://bboxfinder.com/#42.3485,-71.0547,42.3568,-71.0429
[INFO] Using UTM Zone: EPSG:32619
[INFO]
[INFO] Ground Material Type:           Wet Ground       | Frequency Range:   1 - 10  (GHz)
[INFO] Building Rooftop Material Type: Metal            | Frequency Range:   1 - 100 (GHz)
[INFO] Building Wall Material Type:    Concrete         | Frequency Range:   1 - 100 (GHz)
[INFO]
[INFO] Estimated ground polygon size: width=997m, height=902m
Parsing buildings: 100%|████████████████████████| 168/168 [00:00<00:00, 1383.61it/s]
```

Optional flags (v2.0)

bbox finder: https://bboxfinder.com/

Geo2SigMap: Speed Up

Scene generation time comparison: ***Blender-based method*** vs. ***Geo2SigMap***

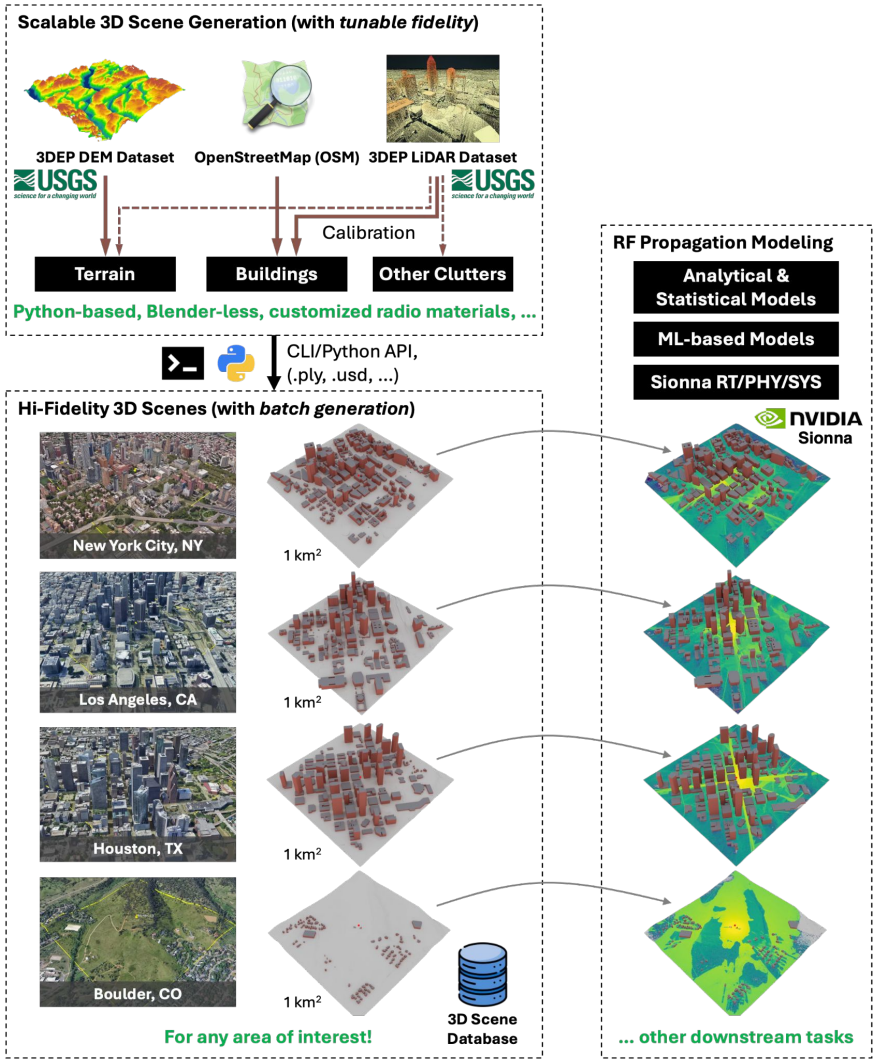


Distribution of 3D scene generation time across 1,000 areas

Geo2SigMap: Summary and Highlights

Release v2.0 features (Nov. 2025)

- Support of lidar datasets
- Support of irregular terrains
- Tunable scene fidelity
- Customized radio materials
- Batch scene generation
- ...



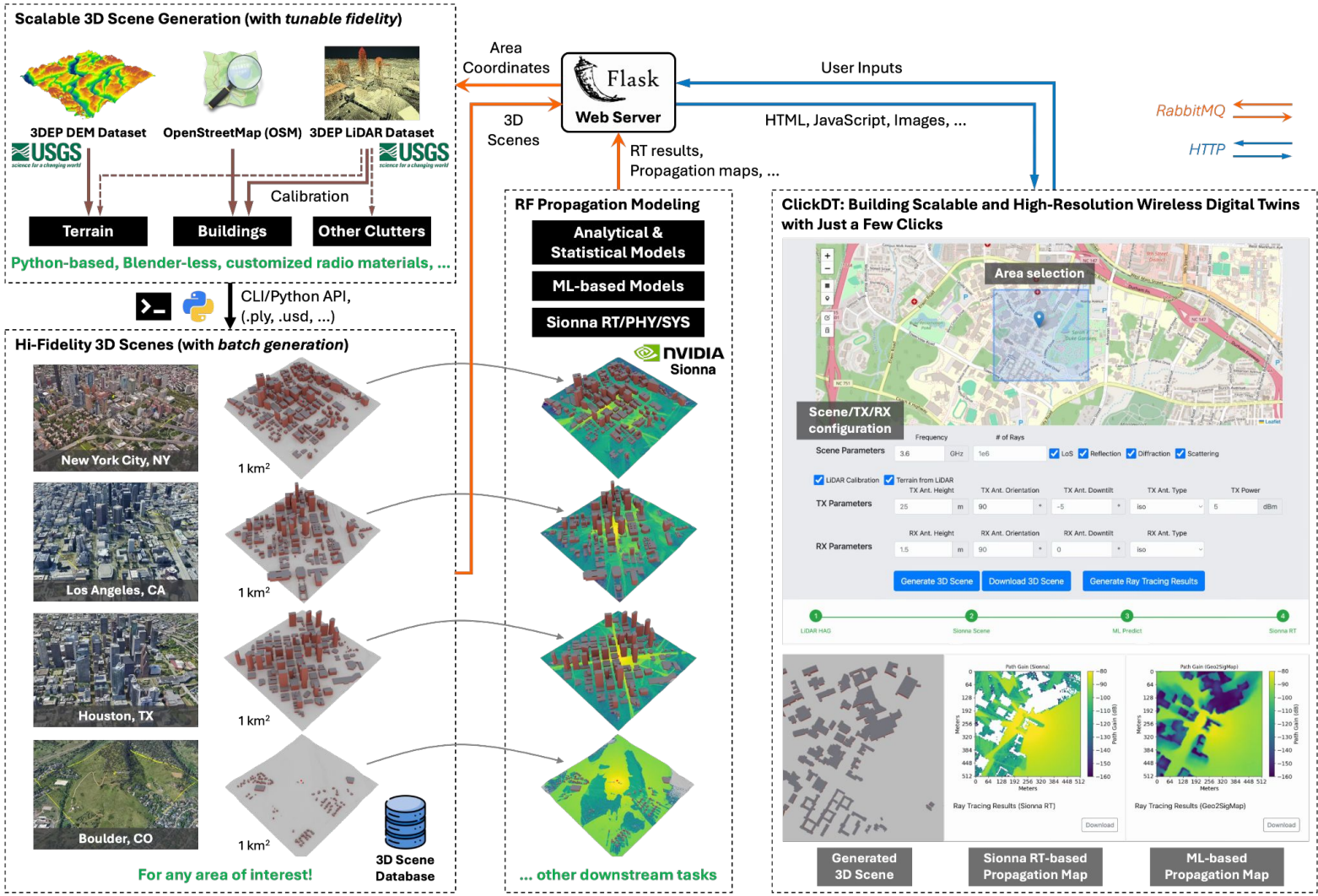
Geo2SigMap: https://github.com/functions-lab/geo2sigmap

Geo2SigMap: Summary and Highlights

Release v2.0 features (Nov. 2025)

- Support of lidar datasets
- Support of irregular terrains
- Tunable scene fidelity
- Customized radio materials
- Batch scene generation
- …

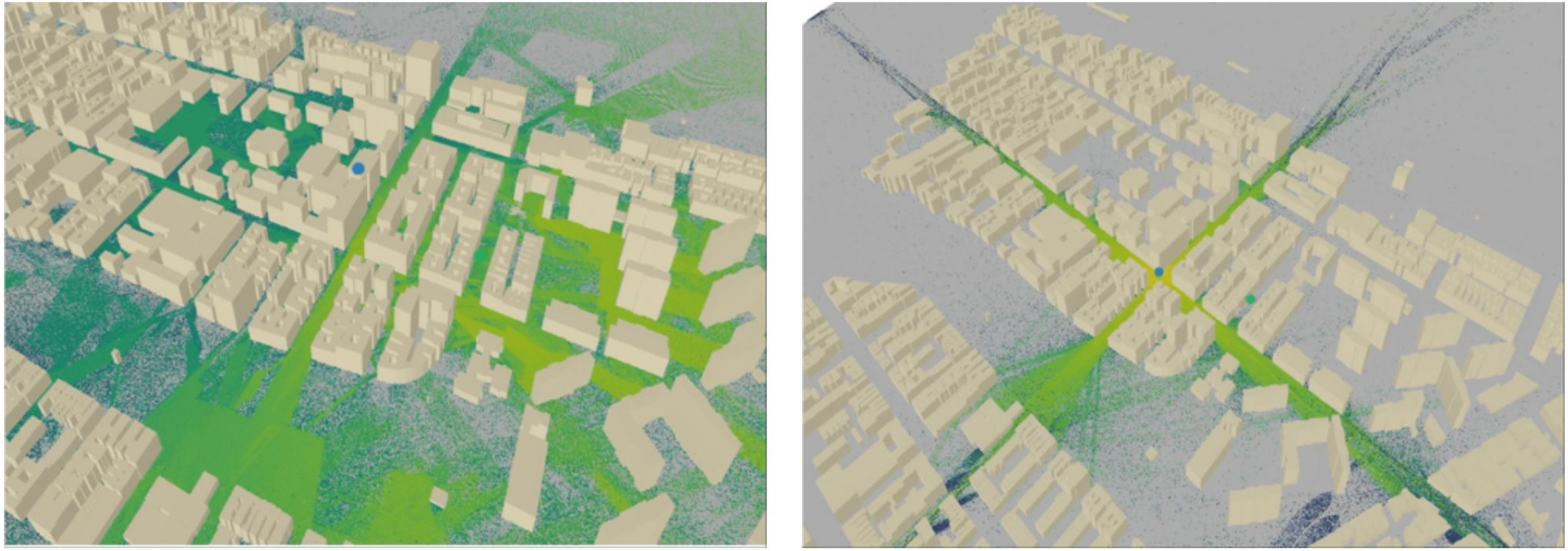
ClickDT: A web-base interface for generating 3D scenes and coverage maps



[1] Y. Li, S. Francini, Z. Gao, and T. Chen, "Demo: ClickDT: Building Scalable and High-Resolution Wireless Digital Twins with a Few Clicks," in Proc. IEEE Military Communications Conference (*MILCOM'25*), 2025.

Wireless DT for the NYC FCC Innovation Zone

~1 sq. mi. area in Upper West, Manhattan, within the PAWR COSMOS platform deployment area, with accurate building height information from a public NYC LiDAR dataset



RF signal coverage for a COSMOS large node in FR1 (2.4 GHz, left) and FR2 (28 GHz, right)

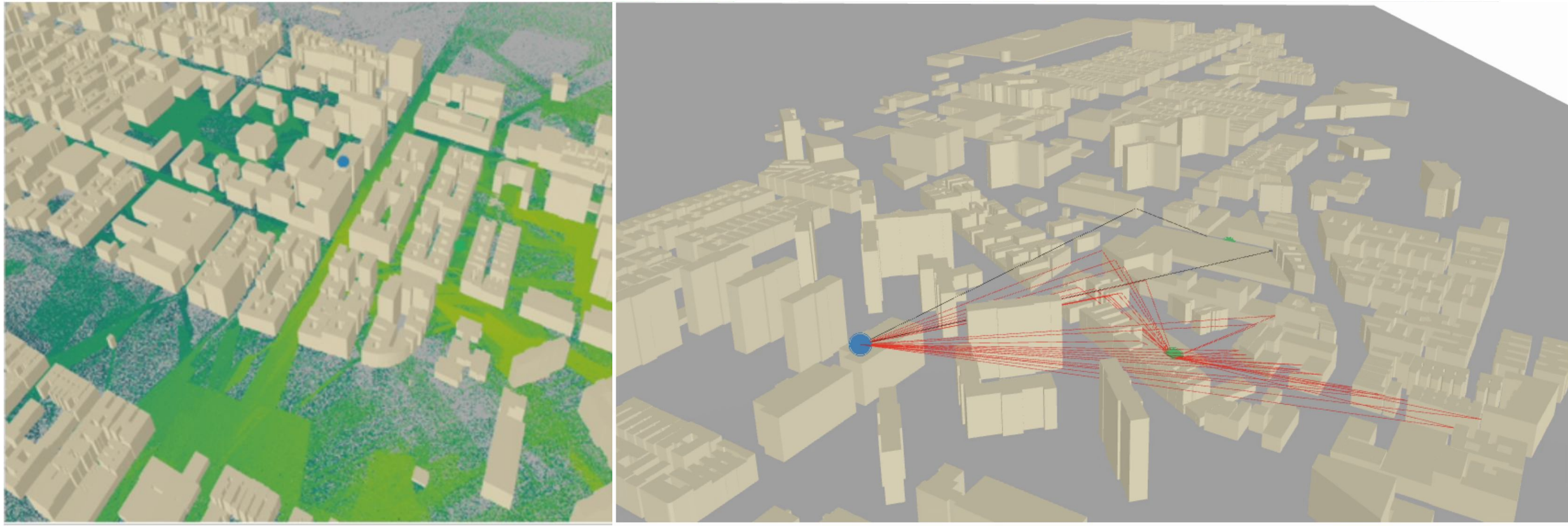
Duke ENGINEERING FuNCtions Lab

PAWR COSMOS testbed: https://www.cosmos-lab.org/
New York State GIS Resources: https://gis.ny.gov/lidar

IEEE MILCOM

Wireless DT for the NYC FCC Innovation Zone

~1 sq. mi. area in Upper West, Manhattan, within the PAWR COSMOS platform deployment area, with accurate building height information from a public NYC LiDAR dataset



RF signal coverage for a COSMOS large node in FR1 (2.4 GHz, left) and FR2 (28 GHz, right)

Part I: Live Coding Time!

You can now begin to work through the following tutorial in Jupyter:

- `0_geo2sigmap_scene_tutorial.ipynb`

Once you are done, you can try the following:

- Generate the 3D scene for any area in the U.S. of your interest ;
  - You can enable/disable the terrain associated with the area
- Visualize the generated 3D scene and compare to ground truth (e.g., Google Maps);

Part II: Site-Specific Radio Map Generation and Ray Tracing (RT) Using Sionna-RT

RF Propagation Models for Wireless DTs

Challenge: Wireless Channels are difficult to estimate in Complex Environments

Analytical/Empirical Models: Easy to compute, but with no site-specific information

- Friis free space path gain model, 3GPP models (e.g., UMa, UMi), Ericsson model, …

Ray Tracing (RT): More computationally expensive, but supports site/radio-specific modeling

- ALTAIR WinProp, REMCOM Wireless InSite, SIRADEL ENGIE Volcano, MathWorks MATLAB, NVIDIA Sionna RT, …

ML-based Models: Fast inference (potentially), but requires large volumes of synthetic a measurement data

- PLNet [*KDD'20*], Geo2SigMap [*DySPAN'24*], …



[1] X. Zhang, X. Shu, B. Zhang, J. Ren, L. Zhou, and X. Chen, "Cellular network radio propagation modeling with deep convolutional neural networks," in Proc. ACM SIGKDD Conference on Knowledge Discovery and Data Mining (*KDD'20*), 2020

Sionna RT: Differentiable Ray Tracing (RT)

Sionna: An open-source Python library for NextG Physical layer research from NVIDIA

Sionna RT: Site-specific radio propagation modeling with accurate channel realizations

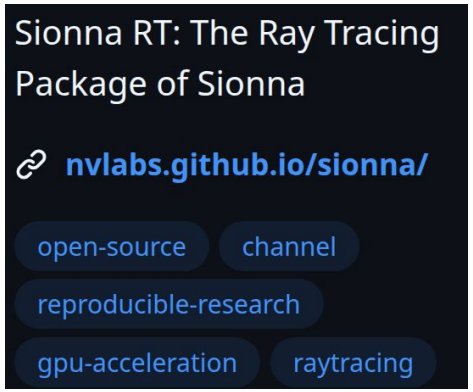
- *Input*: **`{scene, radios}`**
- *Output*: **`{coverage map}`** for the scene, **`{rays}`** for a link, …
- Built on the Mitsuba 3 renderer and Dr.Jit compute engine

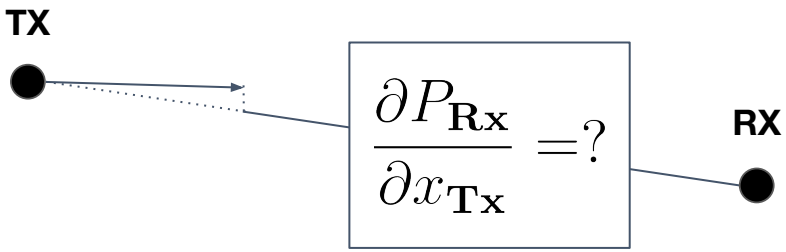


→ Rendering and handling of scenes (e.g., using xml format)
→ Computation of how the rays intersect with scene primitives (e.g., triangles forming a mesh)



→ JIT-compiled vectorized computation
→ Automatic differentiation
→ CPU/GPU acceleration





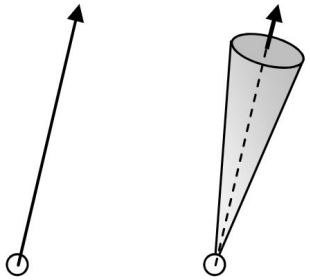
Sionna PHY: Link-level simulator; **Sionna SYS**: System-level simulation

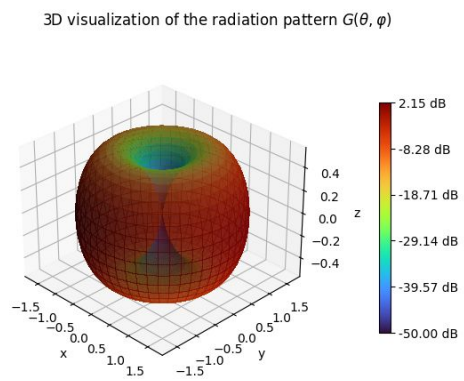
Duke ENGINEERING FuNCtions Lab

Sionna RT: https://github.com/NVlabs/sionna-rt

IEEE MILCOM

Wireless RT: The Basics





Working principles:

- Transmitters (TXs) are modeled as **point sources**
- Wavefronts are represented using lines (**rays**)
- Each ray can be traced through the scene to create **a series of paths**

Each ray carries energy:

- The energy (cross-section) is often modeled as **an expanding "tube"**
- Channel estimates are discretized in space using **2D grids (*Maps*)** or **points (*RXs*)**

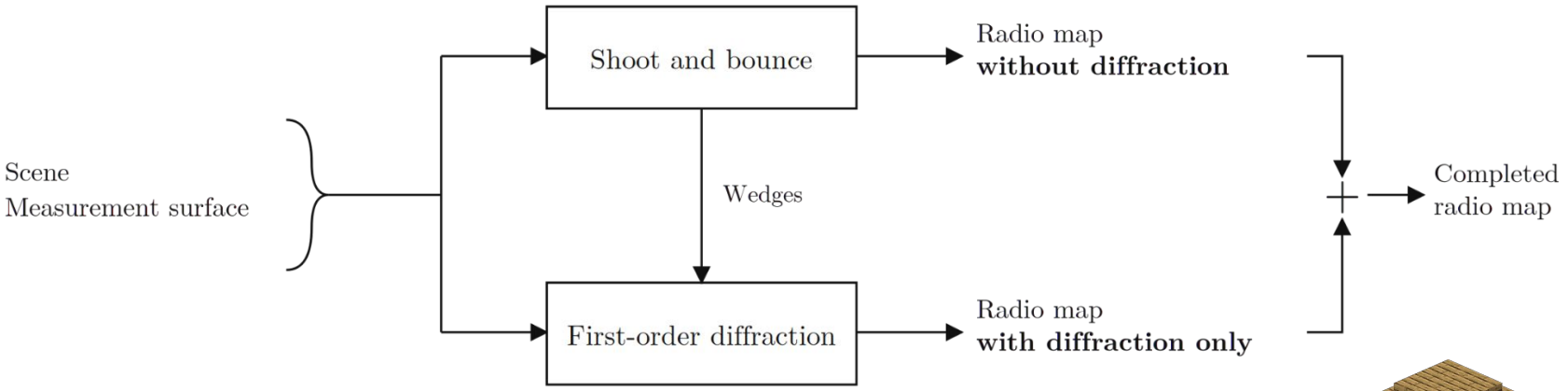
Other considerations:

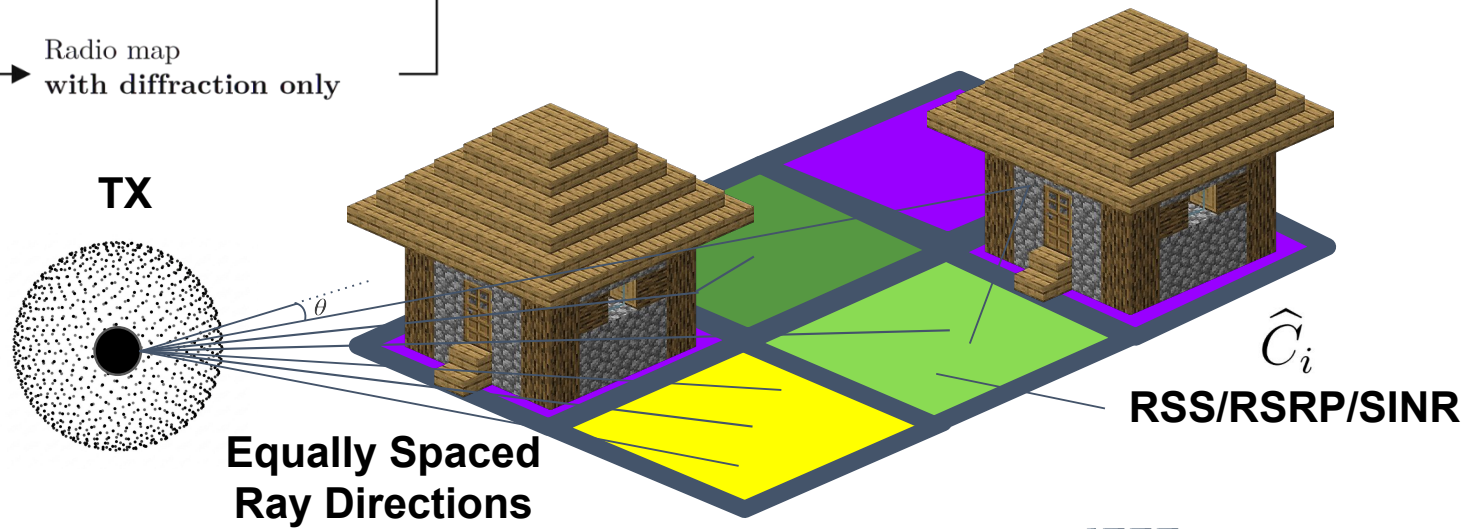
- Each endpoint has an associated **radiation pattern**;
- Every propagation interaction must consider **freq.-dependent material characteristics**
  - Relative permittivity, relative conductivity, …
- RT can be computationally intensive
  - Hardware-accelerated (e.g., using CPUs vs. GPUs) solutions should balance the tradeoff between speed and accuracy

[1] Z. Yun and M. F Iskander, "Ray tracing for radio propagation modeling: Principles and applications," *IEEE Access*, vol. 3, pp. 1089–1100, July 2015.

Wireless RT: The RadioMap Solver

1. Rays are launched uniformly from the TX → **shooting and bouncing of rays (SBR)** method
2. Measurements are discretized into equally sized pixels on a given 2D plane
3. Measurements (visually) represent a spatial average of signal energy
4. Commonly known as a *Radio Map (RM)*, or *Coverage Map*

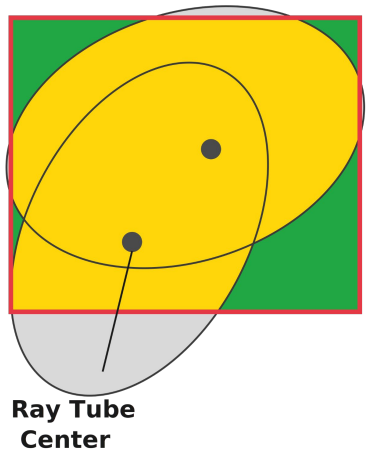




[1] Fayçal Aït Aoudia, Jakob Hoydis, Merlin Nimier-David, Baptiste Nicolet, Sebastian Cammerer, and Alexander Keller. 2025. Sionna RT: Technical Report. arXiv:2504.21719 [cs.IT], 2025. Link: https://arxiv.org/abs/2504.21719

RM Solver: EM Computation Overview

Visualization

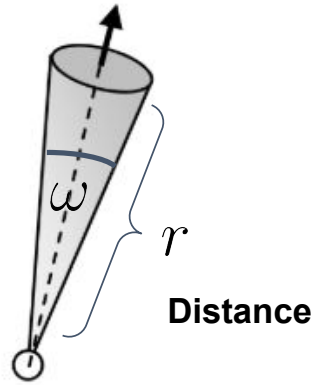


Monte Carlo Estimation:

$$\widehat{C}_i = \frac{1}{|M_i|} \sum_{n=1}^{N_S} \sum_{\ell=0}^{L} e\left(p_n^{(\ell)}\right) \mathbb{I}\left(p_n^{(\ell)}\right) \frac{\beta_n^{(\ell)}}{\Pr\left(\chi_n^{(\ell)}\right)}$$

Where:

$$\beta_n^{(\ell)} = \prod_{i=1}^{n_{\mathrm{rt}}} \frac{r_i^2}{\cos\theta_i} \Delta\omega_i, \quad \Longrightarrow \quad \Delta A = \frac{r^2}{\cos\theta} \Delta\omega$$



Or:

$$\text{Total Signal} = \frac{1}{\text{Pixel Area}} \sum_{\text{All Rays}} \left(\frac{\text{Ray Power} \times \text{Spot Size}}{\text{Sampling Probability}} \right)$$

[1] Fayçal Aït Aoudia, Jakob Hoydis, Merlin Nimier-David, Baptiste Nicolet, Sebastian Cammerer, and Alexander Keller. 2025. Sionna RT: Technical Report. arXiv:2504.21719 [cs.IT], 2025. Link: https://arxiv.org/abs/2504.21719

RM Solver: Code Skeleton

```python
SCENE_FOLDER = "../../scenes/Dupont_flat"
scene = load_scene(f"{SCENE_FOLDER}/scene.xml")

# Set the operating frequency in Hz (here, 3.65 GHz
corresponds to the CBRS band)
scene.frequency = 3.65e9

# Create a Planar Array with 1 element
scene.tx_array = PlanarArray(
    # Array size
    num_rows=1,
    num_cols=1,
    # Wavelength normalized spacing
    vertical_spacing=0.5,
    horizontal_spacing=0.5,
    # Antenna parameters
    pattern="dipole",
    polarization="V"
)
```



```python
# Create a single transmitter (tx0)
tx0 = Transmitter(
    name="tx",
    position=[-5, 5, 120]
)

# Add the transmitter instance to the scene
scene.add(tx0)

rm_solver = RadioMapSolver() # Create a RM Solver Object

rm = rm_solver(scene,
    max_depth=5,            # Max # of scene interactions
    los=True,               # LOS paths considered
    refraction=False,       # Interaction(s)
    cell_size=(1., 1.),     # Pixel Size (grid)
    samples_per_tx=int(1e8) # Max sample count of rays
)
```

Duke ENGINEERING FuNCtions Lab

Sionna RT: https://github.com/NVlabs/sionna-rt

IEEE MILCOM

Part II(a): Live Coding Time!

You can now begin to work through the following tutorial in Jupyter:

- `1_sionna_coverage_map.ipynb`

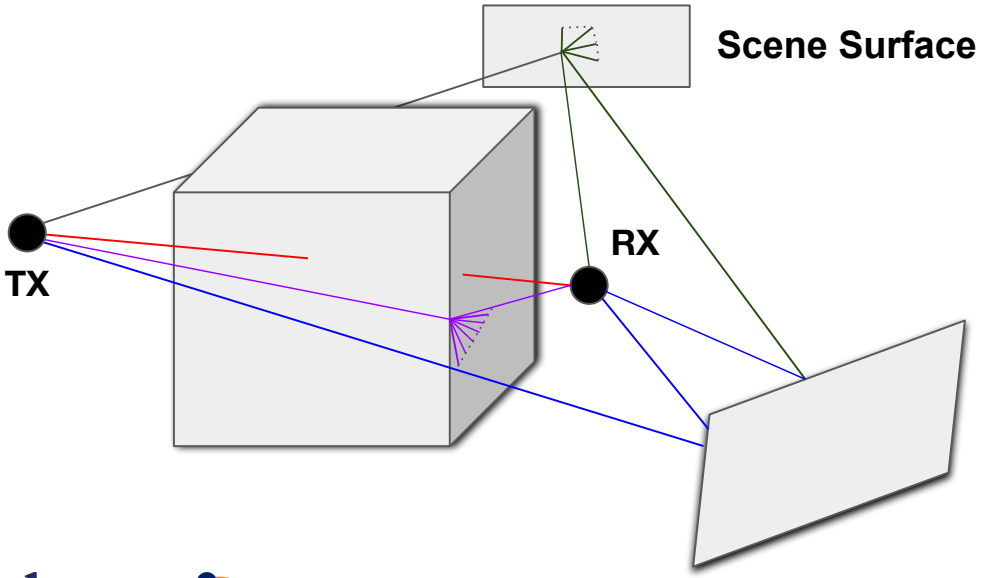
Once you are done, you can try the following:

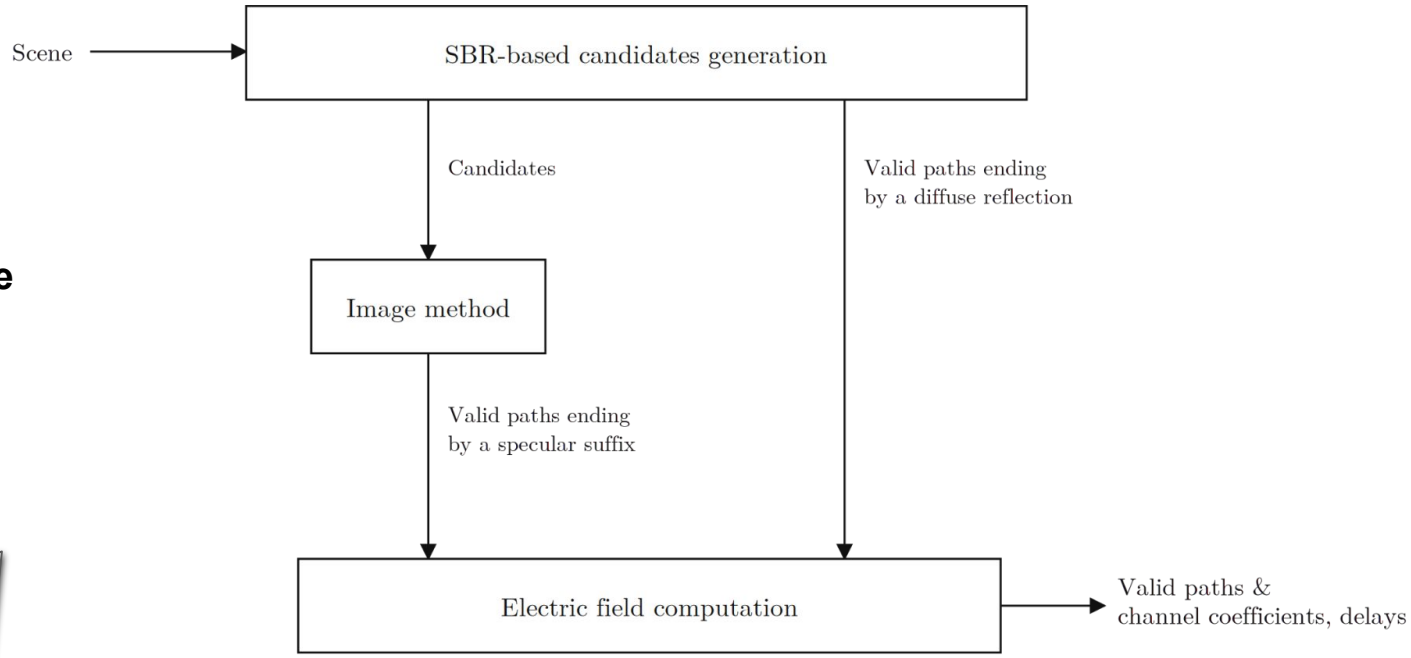
- Generate radio map(s) for the 3D scene(s) that you just created
- Modify the transmitter settings (e.g., location) and RadioMapSolver settings (e.g., cell size, interactions, # of rays) and see how affect the generated radio maps

Coffee Break

Wireless RT: The PathSolver Method

1. Rays are launched from the TX using **the SBR method**;
2. Connecting paths are isolated using **the image method**;
   a. Virtual sources are used to validate TX-RX connections
3. Colored rays denote path types:
   a. — Specular reflection
   b. — Diffuse reflection (scattering)
   c. — Refraction
   d. — Diffraction





PathSolver: The point-to-point method

[1] Fayçal Aït Aoudia, Jakob Hoydis, Merlin Nimier-David, Baptiste Nicolet, Sebastian Cammerer, and Alexander Keller. 2025. Sionna RT: Technical Report. arXiv:2504.21719 [cs.IT], 2025. Link: https://arxiv.org/abs/2504.21719

PathSolver: EM Computation Overview

Channel coefficients:

$$a_n = \frac{\lambda}{4\pi} \mathbf{C}_{\mathrm{R}}(\theta_{\mathrm{R},n}, \varphi_{\mathrm{R},n})^{\mathsf{H}} \left(\prod_{\ell'=1}^{\ell} \mathbf{T}_n^{(\ell')} \right) \mathbf{C}_{\mathrm{T}}(\theta_{\mathrm{T},n}, \varphi_{\mathrm{T},n})$$

Channel impulse response (CIR):

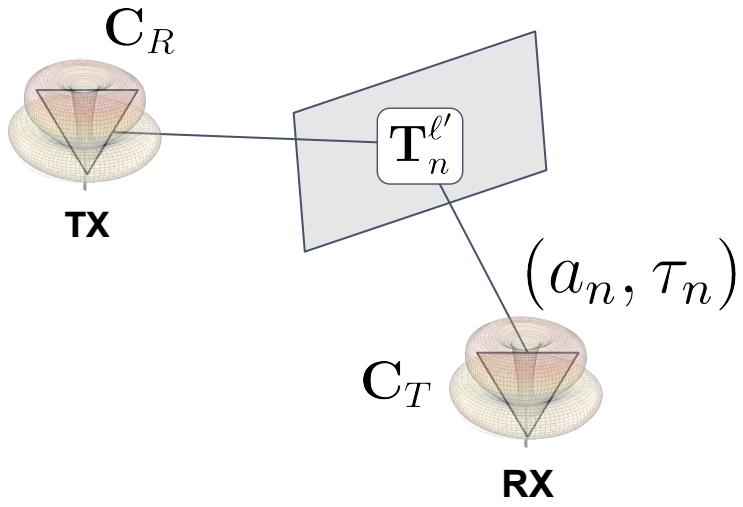
$$h(\tau) = \sum_{n=1}^{N} a_n \delta(\tau - \tau_n)$$

Or:

$$\text{Channel Response} = \sum_{\text{All Paths}} (\text{Path Strength}) \text{ at its arrival time}$$

Where:

a_n : path coefficient
τ_n : propagation delay
$\mathbf{C}_{\mathrm{T}}, \mathbf{C}_{\mathrm{R}}$: Tx/Rx antenna patterns
$\mathbf{T}_n^{(\ell')}$: interaction matrix



[1] Fayçal Aït Aoudia, Jakob Hoydis, Merlin Nimier-David, Baptiste Nicolet, Sebastian Cammerer, and Alexander Keller. 2025. Sionna RT: Technical Report. arXiv:2504.21719 [cs.IT], 2025. Link: https://arxiv.org/abs/2504.21719

PathSolver: Code Skeleton

```python
# Define Receive Array (single dipole element)
scene.rx_array = PlanarArray(
    # Array size
    num_rows=1,
    num_cols=1,
    # Wavelength normalized spacing
    vertical_spacing=0.5,
    horizontal_spacing=0.5,
    # Antenna parameters
    pattern="dipole",
    polarization="V"
)
# Create and add a Receiver to the Scene
rx = Receiver(
        name="rx",
        position=[0, 0, 1.5],
        orientation=[0, 0, 0])
```



```python
# Add the receiver to the scene
scene.add(rx)

# Create an instance of the PathSolver Class
solver = PathSolver()

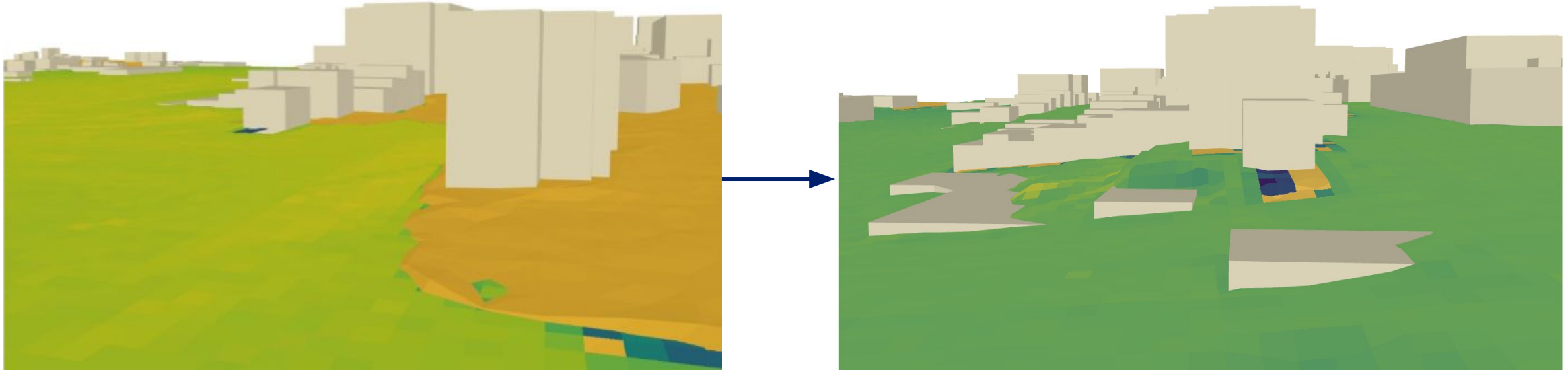
# Generate Paths object
paths = solver(
     scene,                          # Scene Object
     max_depth=3,                    # Max interaction #
     los=True,                       # LOS considered
     specular_reflection=True,       # Interaction(s)
     samples_per_src=int(1e8)        # Ray samples per src
)

# Preview Scene using paths
scene.preview(paths=paths);
```

Advanced Use Cases

Sionna RT supports **irregular measurement planes** (starting v2.0.0):

- Instead of using a 2D grid for channel measurements, one can also characterize radio wave propagation using the lidar terrain mesh triangles.
- Our notebook example, **`1_sionna_coverage_map.ipynb`**, adapts this feature to quantify coverage along the rough terrain.



Advanced Use Cases

Ground material definitions can be characterized for **extended frequency ranges**:

- Out of the box, Sionna RT uses ITU P.2040-3 which supports simulations from *1–10 GHz*.
  Ref: https://nvlabs.github.io/sionna/rt/api/radio_materials.html#
- Geo2SigMap support extended frequency ranges using ITU-R P.527-3 to *1 MHz–100 GHz*.
  An example will be showcased in Part III: **`5_scene_nist_28ghz_boulder.ipynb`**

Sionna RT also supports **mobile scenarios**:

- Doppler shifts can be modeled over both short/long time horizons
- Useful for generating time-varying CIRs
- Ref: https://nvlabs.github.io/sionna/rt/tutorials/Mobility.html

Part II(b): Live Coding Time!

You can now begin to work through the following tutorial in Jupyter:

- `2_sionna_ray_analysis.ipynb`

Once you are done, you can try the following:

- Generate rays characteristics for the 3D scene(s) that you just created
- Modify the transmitter and receiver settings (e.g., location) and PathSolver settings (e.g., interactions, # of rays) and see how affect the generated rays
- Perform ray analysis, including the advance options (e.g., channel frequency response)

Part III: Connecting Wireless DTs with Real-World Measurements

Evaluating the *Sim2Real* Gap

Question: With ray tracing data widely accessible, how accurate are these results?

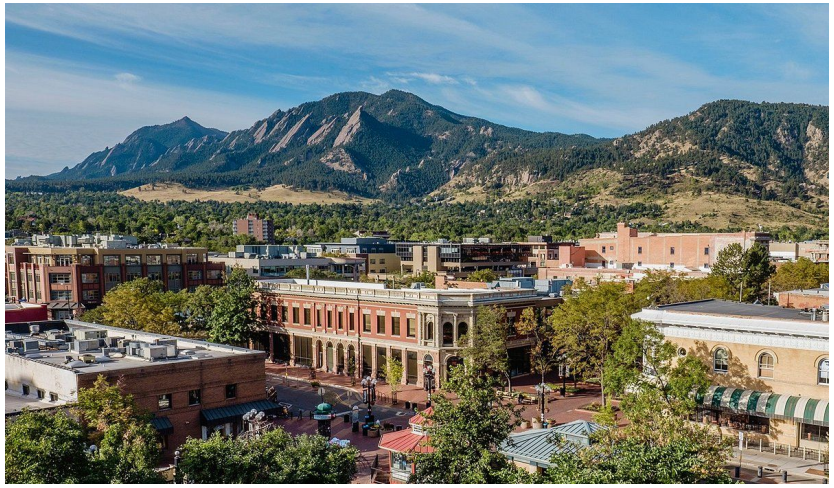
- Wireless DTs can also be used to generate large-scale synthetic datasets → Real-world measurements are the key for validating, calibrating, and fine-tuning the DT simulations

We evaluate and compare the accuracy of **(vanilla) RT vs. real-world measurements**:

- RSRP and coverage (*using CBRS measurements from Duke campus, Durham, NC*)
- Channel gain coefficients, power delay profile (PDP), AoA/AoD (*using 28 GHz measurements from Boulder, CO*)



Durham, NC (Duke U.): 3.55-3.7 GHz

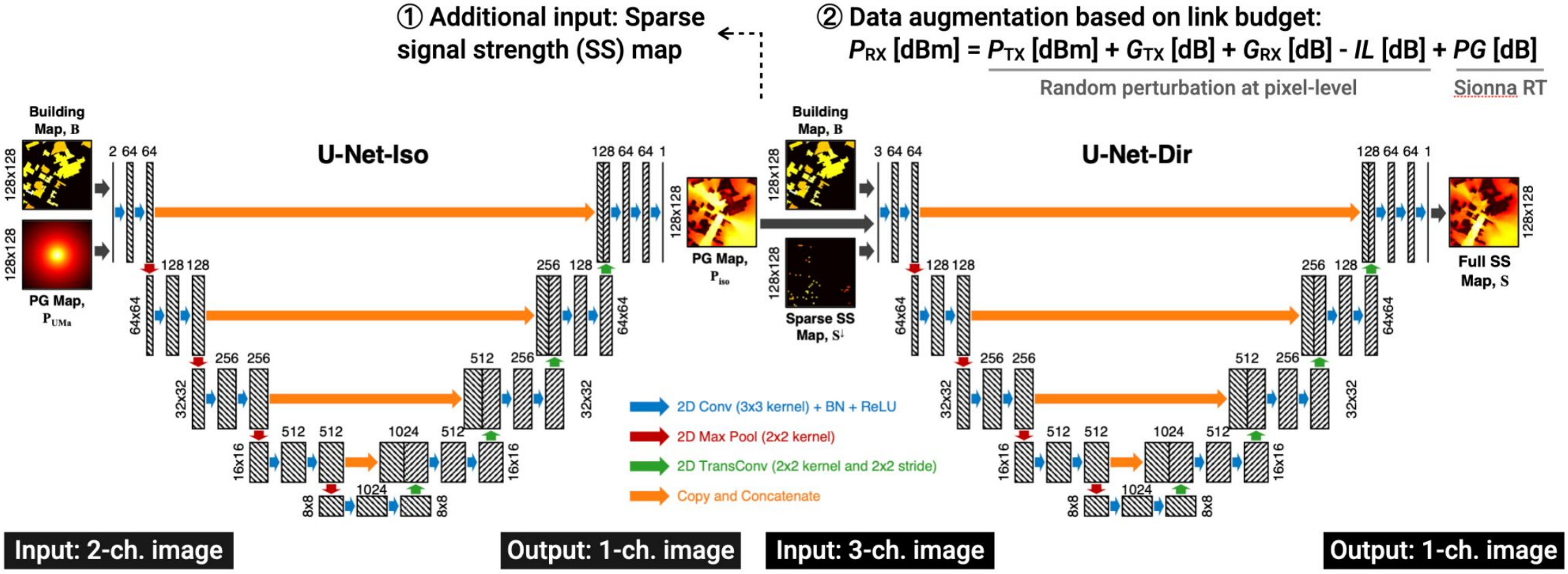


Boulder, CO: 28 GHz

Geo2SigMap's DL Model

The model is ***trained on pure synthetic data*** and ***evaluated by real-world measurements***:

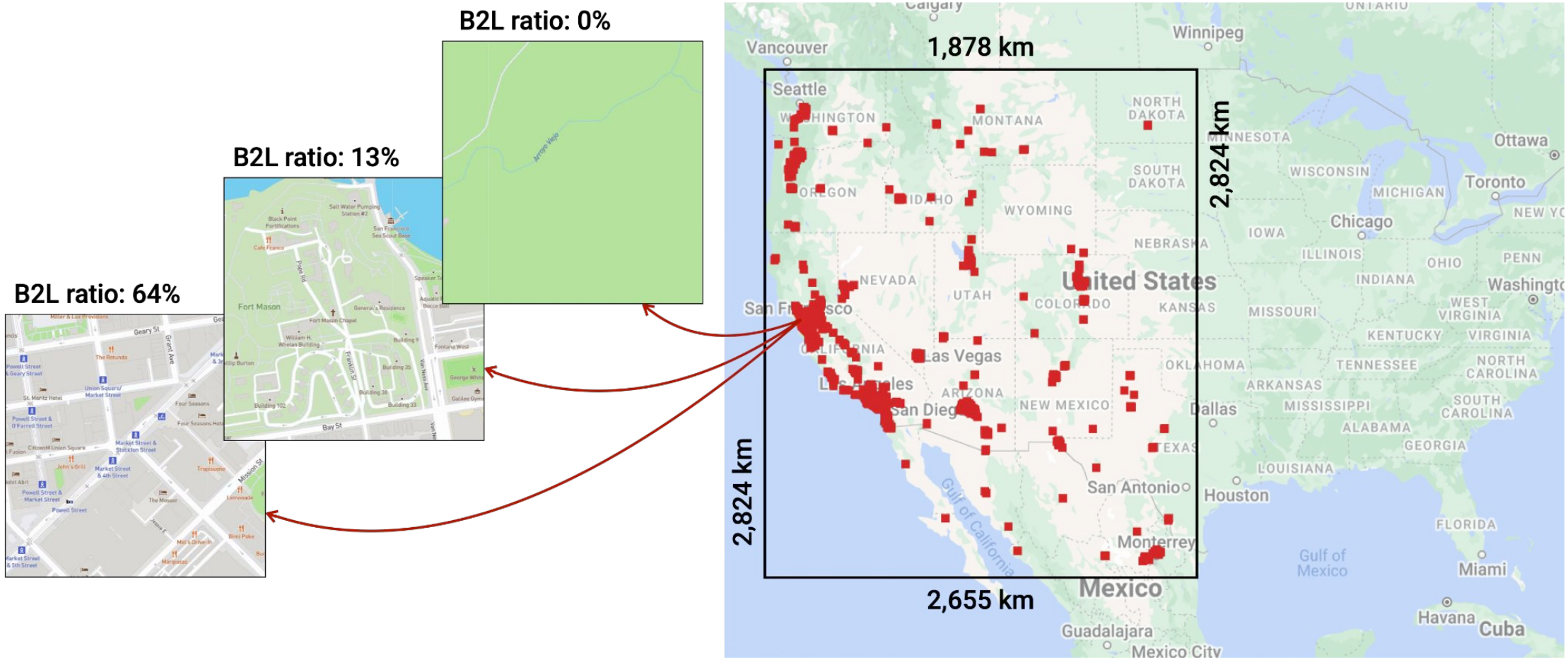
- 1st coarse-grained U-Net (U-Net-Iso), followed by a 2nd fine-grained U-Net (U-Net-Dir)



[1] Y. Li, Z. Li, Z. Gao, and T. Chen, "Geo2SigMap: High-Fidelity RF Signal Mapping Using Geographic Databases," in Proc. IEEE International Symposium on Dynamic Spectrum Access Networks (*DySPAN'24*), 2024.

Dataset Generation: Area Selection

- Divide the “west half” of the U.S. into areas with an area size of 512m x 512m
- Building-to-land (B2L) ratio of >20% → **~27K selected areas**, covering ~7,142 km^2 of land

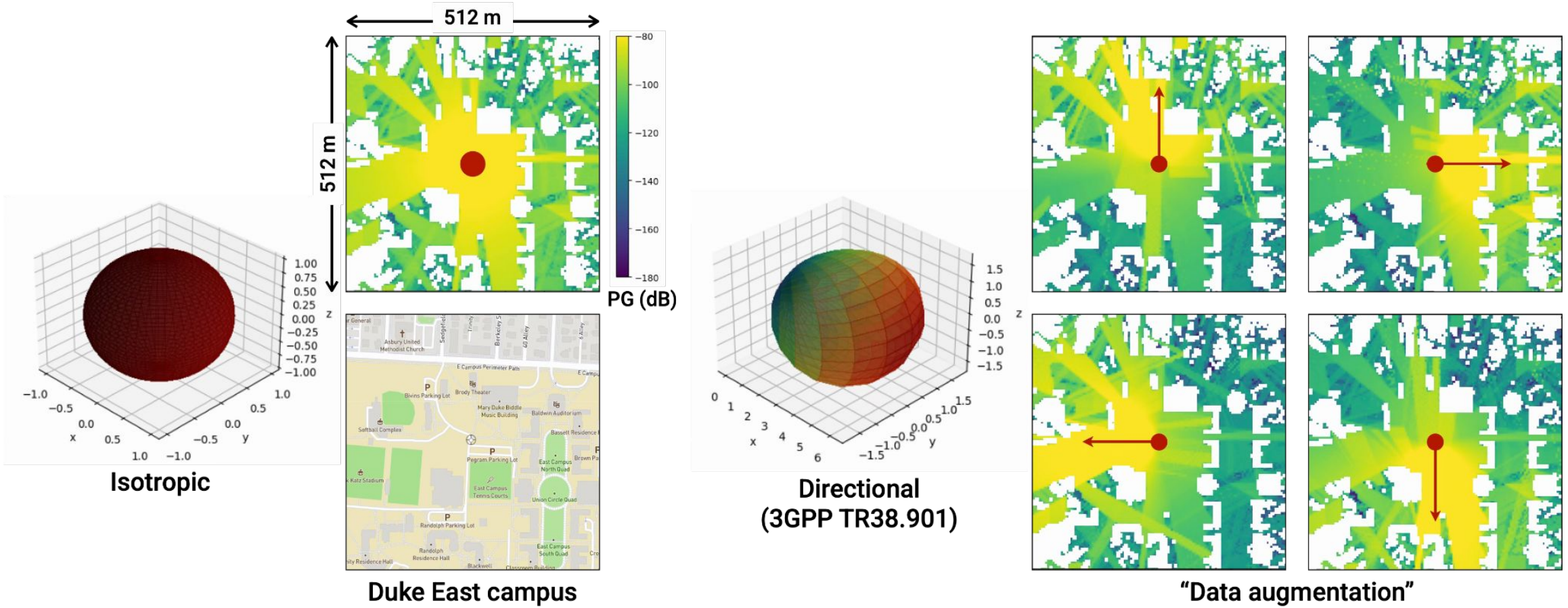


[1] Y. Li, Z. Li, Z. Gao, and T. Chen, “Geo2SigMap: High-Fidelity RF Signal Mapping Using Geographic Databases,” in Proc. IEEE International Symposium on Dynamic Spectrum Access Networks (*DySPAN’24*), 2024.

Dataset Generation: Path Gain (PG) Maps

Five PG maps are generated for each area using Sionna RT:

- 1x with isotropic antenna, 4x with directional antenna



[1] Y. Li, Z. Li, Z. Gao, and T. Chen, "Geo2SigMap: High-Fidelity RF Signal Mapping Using Geographic Databases," in Proc. IEEE International Symposium on Dynamic Spectrum Access Networks (*DySPAN'24*), 2024.

Geo2SigMap's DL Model: Training Set

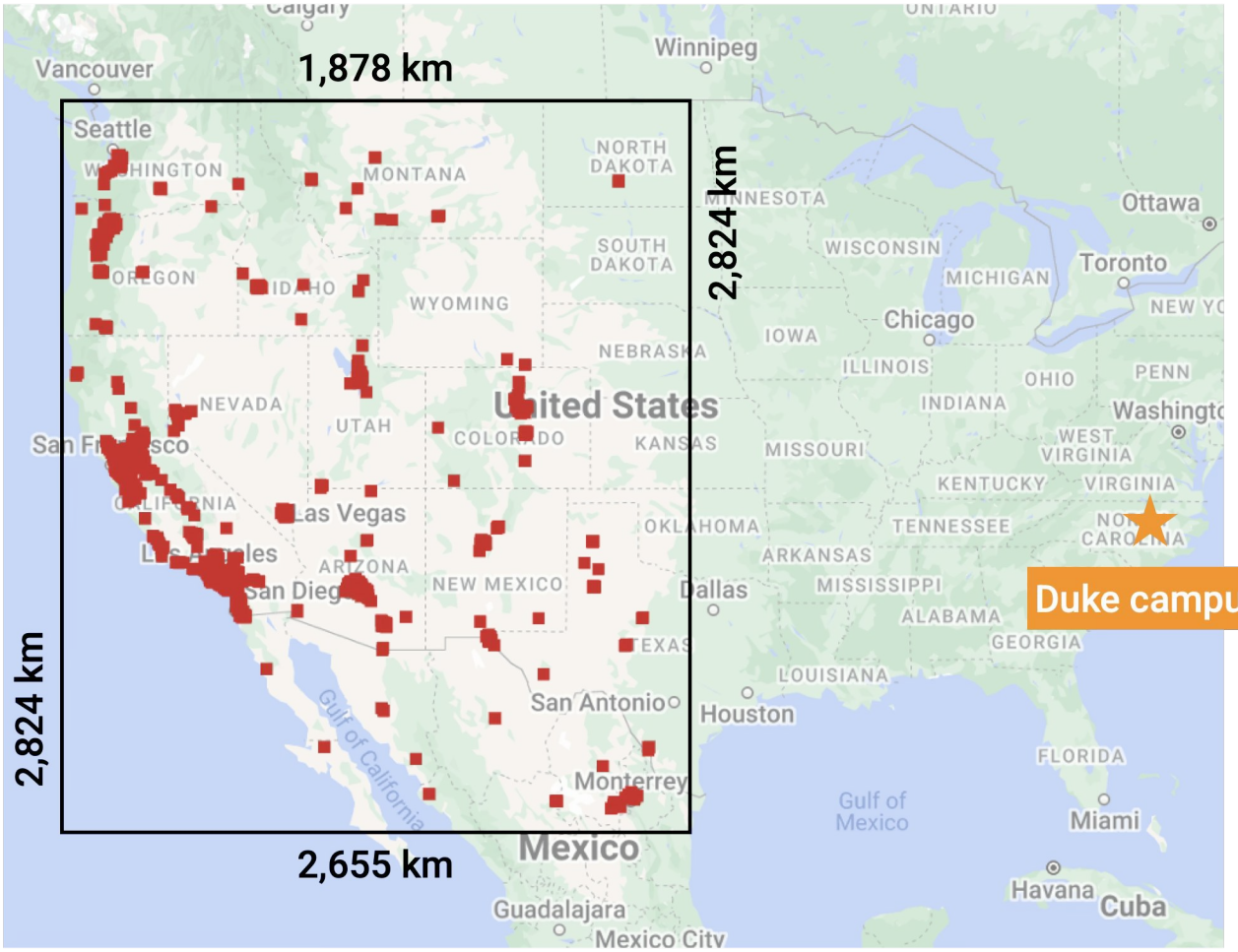
No overlapped areas between ***training set*** (RT simulation) and ***testing set*** (measurements)!

Model training:

- Dataset: ~27K areas, ~140K PG maps
- Loss function: MSE between the ground truthand predicted PG maps
- Training/validation split ratio of 0.8/0.2
- Model size: ~31M params

Model evaluation:

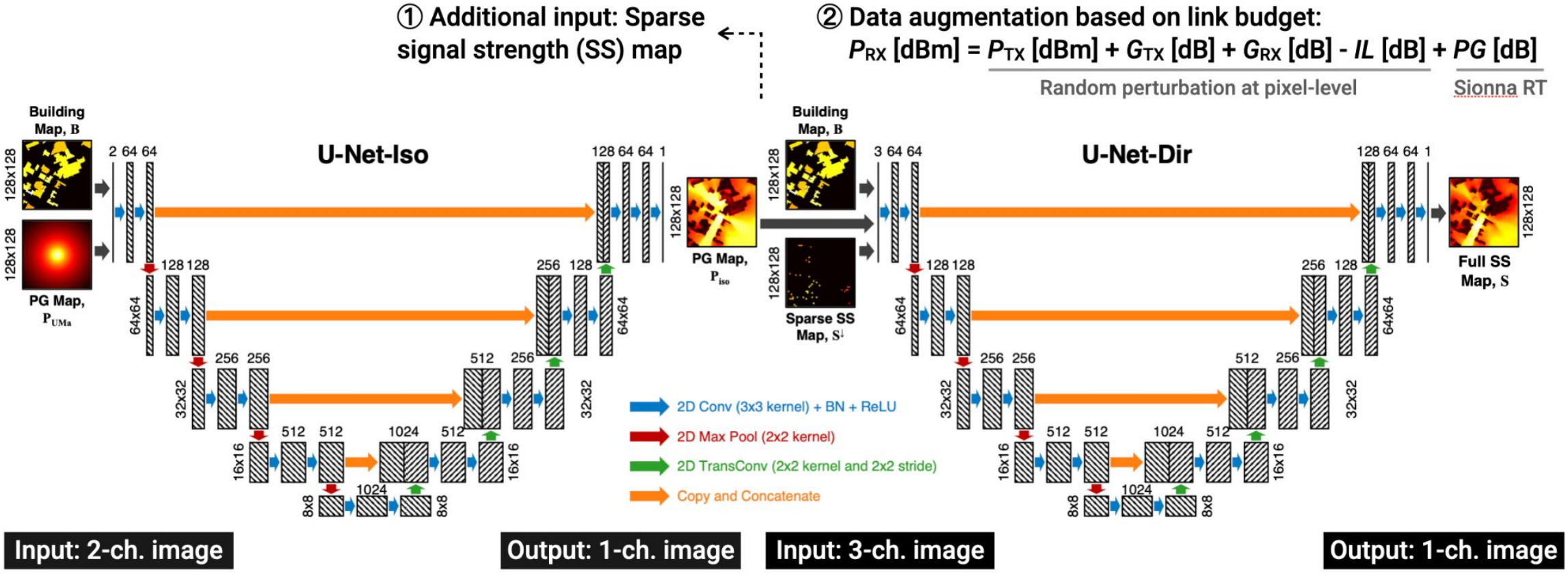
- Dataset: ~57K CBRS meas. collected over 6 months on the Duke campuses



Geo2SigMap's DL Model

The model is ***trained on pure synthetic data*** and ***evaluated by real-world measurements***:

- 1st coarse-grained U-Net (U-Net-Iso), followed by a 2nd fine-grained U-Net (U-Net-Dir)



[1] Y. Li, Z. Li, Z. Gao, and T. Chen, "Geo2SigMap: High-Fidelity RF Signal Mapping Using Geographic Databases," in Proc. IEEE International Symposium on Dynamic Spectrum Access Networks (*DySPAN'24*), 2024.

CBRS Measurements (Duke U.)



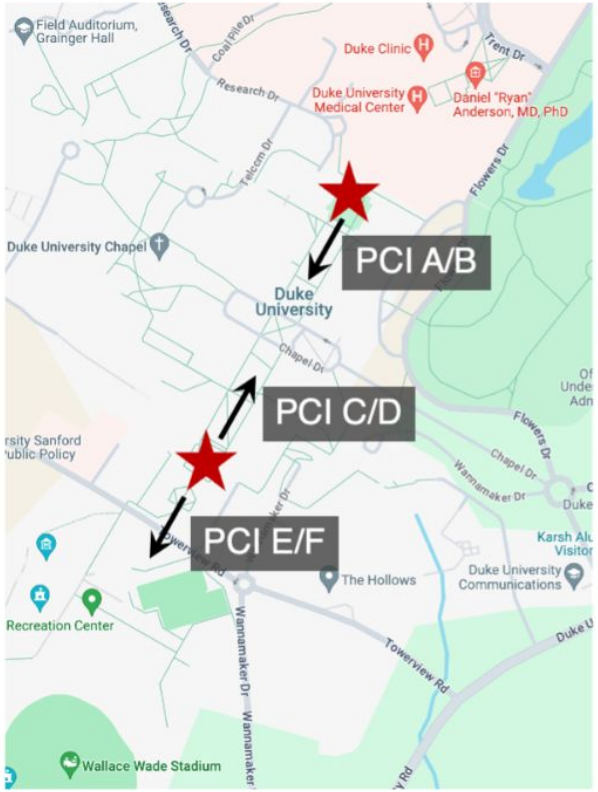
AirSpeed 1030

Six private LTE cells deployed on the Duke West campus

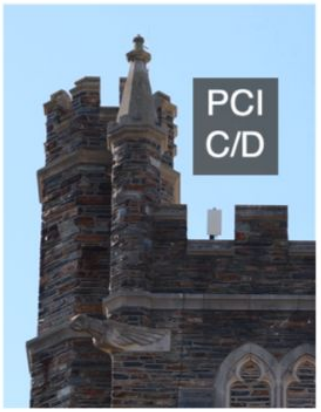
- CBRS band (3.55–3.7 GHz), 20 MHz bandwidth

PCI	Frequency	Bandwidth	Height	Azimuth	Downtilt
A	3.69 GHz	20 MHz	84.6 ft	216°	10°
B	3.64 GHz	20 MHz	84.6 ft	216°	10°
C	3.58 GHz	20 MHz	99.6 ft	30°	18°
D	3.56 GHz	20 MHz	99.6 ft	30°	18°
E	3.69 GHz	20 MHz	99.6 ft	212°	16°
F	3.64 GHz	20 MHz	99.6 ft	212°	16°
G	3.69 GHz	20 MHz	82.5 ft	127°	10°

*PCI (Physical cell ID) has been anonymized









Geo2SigMap: https://github.com/functions-lab/geo2sigmap

CBRS Measurements (Duke U.)

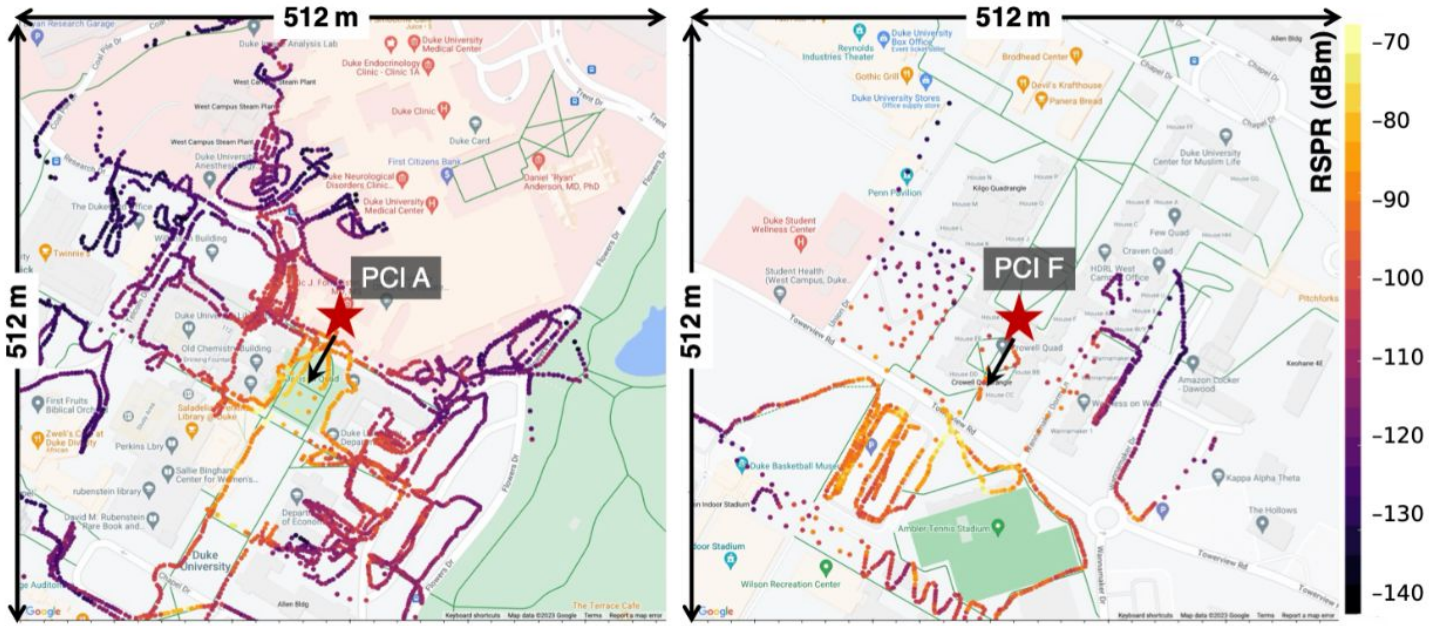
Customized measurement apps and scripts

- Location info (e.g., device GPS and orientation)
- LTE cell info (e.g., MCC, MNC, band #, ...)
- LTE signal KPIs (e.g., RSRP, RSSI, RSSQ, ...)



Galaxy A42 RPi w/ LTE HAT Pixel 5

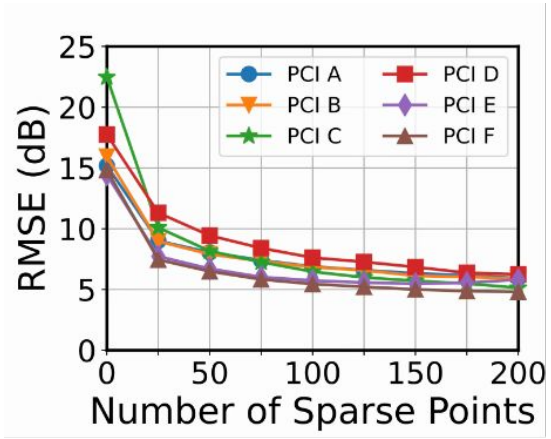
PCI	Samsung Galaxy A42	Google Pixel 5	RPi w/ LTE HAT	Total
A	4,297	3,975	4,110	**12,382**
B	3,461	3,425	8,496	**15,355**
C	1,599	1,423	3,741	**6,763**
D	1,436	1,327	4,425	**7,188**
E	1,185	1,264	1,792	**4,241**
F	1,298	1,325	3,915	**6,538**
G	1,256	1,559	1,406	**4,221**
Total	**14,532**	**14,298**	**27,858**	**56,688**

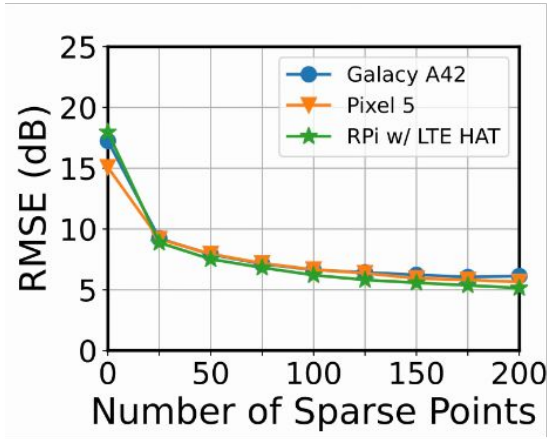


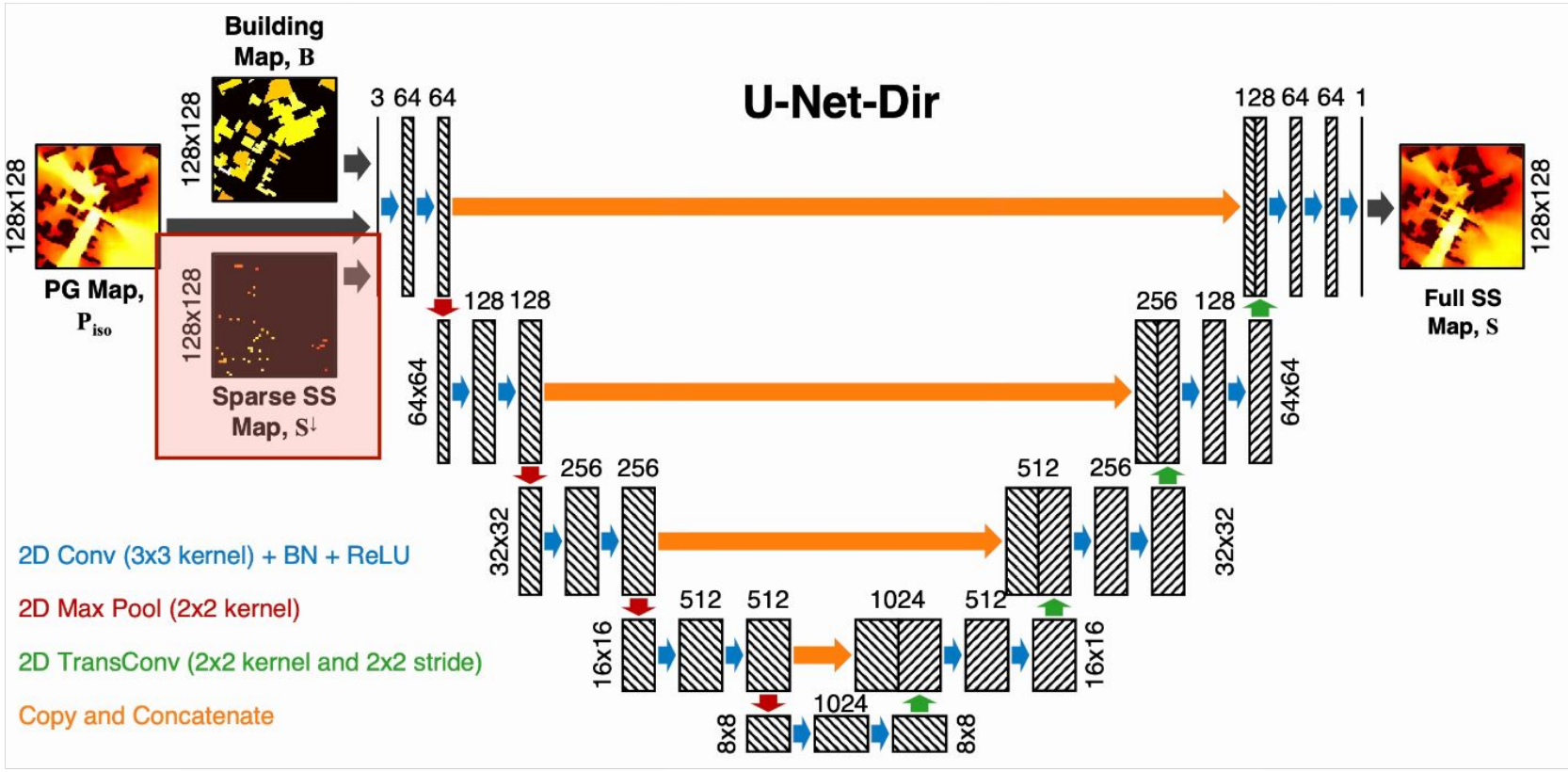
Geo2SigMap: https://github.com/functions-lab/geo2sigmap

of Sparse Measurement Points Needed?

Varying # of sparse points input to the pre-trained model during inference: **~100 sparse points** can achieve a good balance between measurement efforts and prediction accuracy





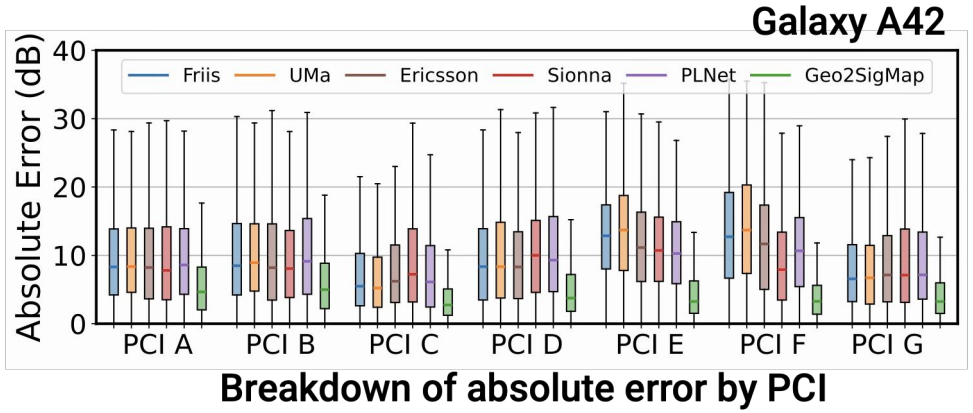


[1] Y. Li, Z. Li, Z. Gao, and T. Chen, "Geo2SigMap: High-Fidelity RF Signal Mapping Using Geographic Databases," in Proc. IEEE International Symposium on Dynamic Spectrum Access Networks (*DySPAN'24*), 2024.

Geo2SigMap: Overall Performance

Baselines (w/ calibration applied)

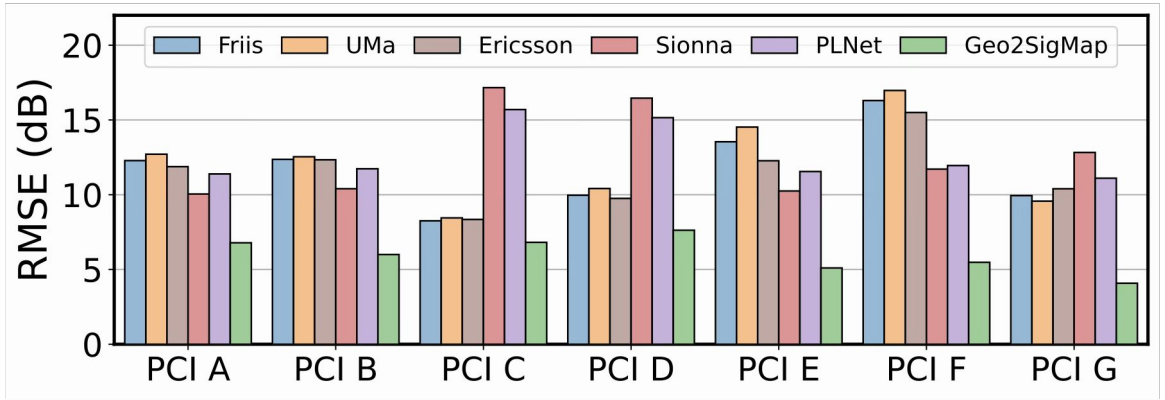
- **Friis** free space PG model
- 3GPP urban macro (**UMa**) channel model
- **Ericsson** channel model
- "Vanilla" **Sionna RT**
- **PLNet** with model re-training [*KDD'20*]



Breakdown of absolute error by PCI

Metric: Root mean square error (RMSE) for RSRP

- Breakdown by PCIs and UE types



Geo2SigMap achieves an RMSE of 7.1 dB for predicting RF signal coverage, outperforming existing methods by 4.0 dB

[1] Y. Li, Z. Li, Z. Gao, and T. Chen, "Geo2SigMap: High-Fidelity RF Signal Mapping Using Geographic Databases," in Proc. IEEE International Symposium on Dynamic Spectrum Access Networks (*DySPAN'24*), 2024.

Geo2SigMap: Overall Performance

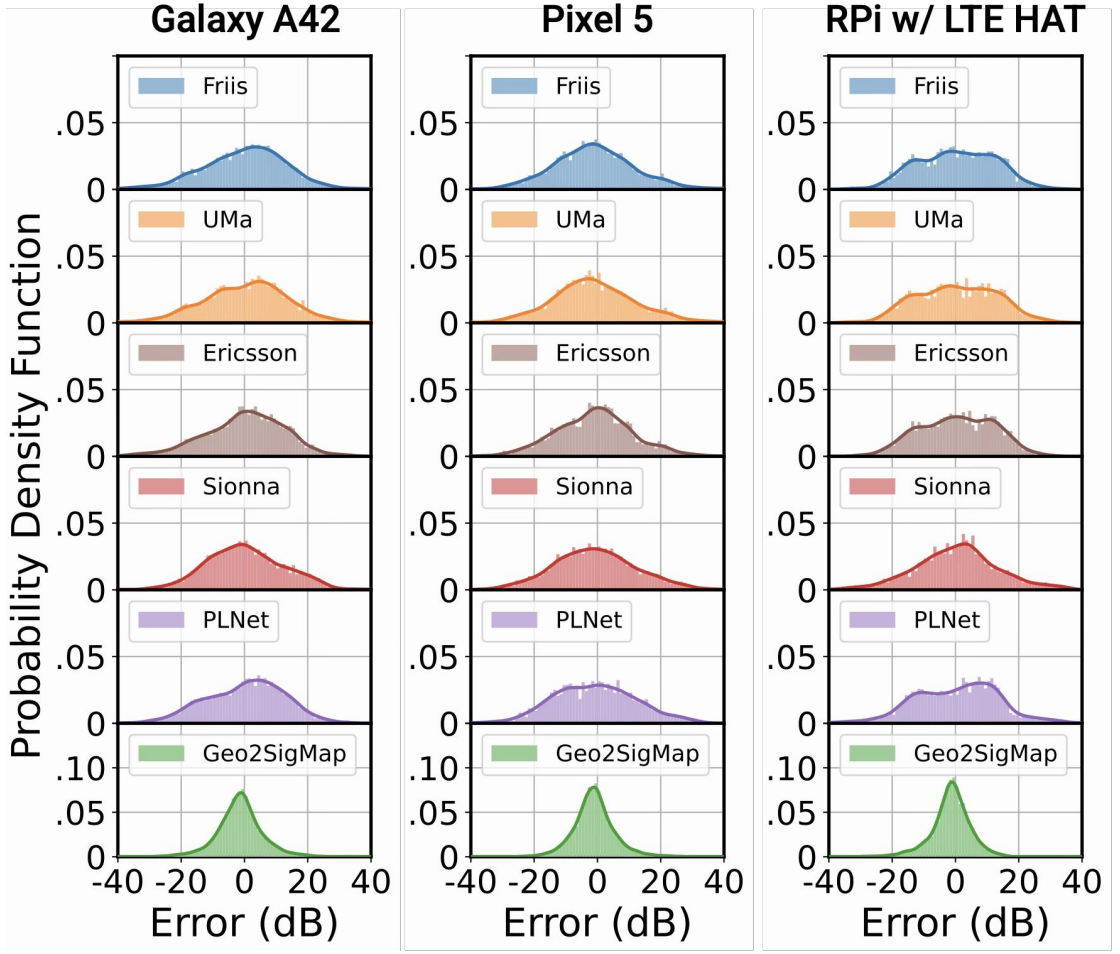
Baselines (w/ calibration applied)

- **Friis** free space PG model
- 3GPP urban macro (**UMa**) channel model
- **Ericsson** channel model
- "Vanilla" **Sionna RT**
- **PLNet** with model re-training [*KDD'20*]

Metric: Root mean square error (RMSE) for RSRP

- Breakdown by PCIs and UE types

Geo2SigMap also achieves a more concentrated error distribution across PCIs and UE types



[1] Y. Li, Z. Li, Z. Gao, and T. Chen, "Geo2SigMap: High-Fidelity RF Signal Mapping Using Geographic Databases," in Proc. IEEE International Symposium on Dynamic Spectrum Access Networks (*DySPAN'24*), 2024.

Part III(a): Live Coding Time!

You can now begin to work through the following tutorial in Jupyter:

- `3_visualize_measurements.ipynb`
- `4_ml_coverage_map.ipynb`

28 GHz Measurement (Boulder, CO)

NIST NextG channel measurement project

- Downtown Boulder, using a channel sounder
- Fixed TX + moving RX on a ground robot
- Environment/transmission characteristics are defined for adaptations to RT





NIST 28 GHz measurement dataset: https://nextg.nist.gov/submissions/112

28 GHz Measurement (Boulder, CO)

NIST NextG channel measurement project

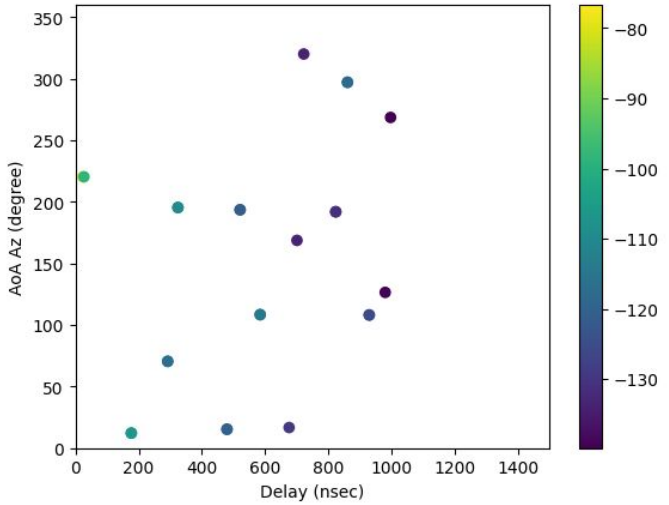
- Downtown Boulder, CO, using a channel sounder
- Fixed TX + moving RX on a ground robot
- Environment/transmission characteristics are defined for adaptations to RT



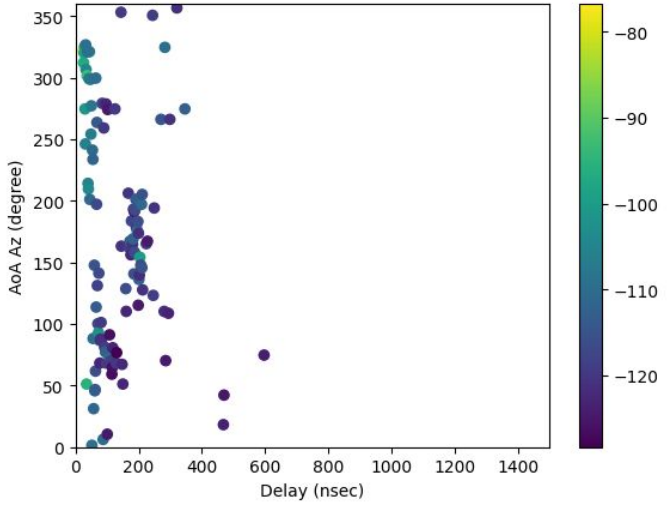
LOS Path

NLOS path

LOS, Sionna RT



LOS, Measurements



NIST 28 GHz measurement dataset: https://nextg.nist.gov/submissions/112

Part III(b): Live Coding Time!

You can now begin to work through the following tutorial in Jupyter:

- `5_scene_nist_28ghz_boulder.ipynb`

Thank You for Attending!

Summary: A practical, hands-on tutorial to developing *scalable*, *physically-accurate*, and *high-fidelity* wireless digital twins (DTs), with three components: **Geo2SigMap**, **NVIDIA Sionna RT**, and **Closing the *sim2real* Gap** (with measurements)

Contents: 3-part lecture + 6 hands-on Jupyter notebooks,

→ *Future editions are under preparation and suggestions are welcome!*

Contact info: Tingjun Chen (tingjun.chen@duke.edu),
Yiming Li (yiming.li@duke.edu), Sam Rivera (sam.rivera@duke.edu)

Duke ENGINEERING FuNCtions Lab

Platforms for Advanced Wireless Research

COSMOS

ATHENA NSF AI Institute for Edge Computing

CUbiC Center for Ubiquitous Connectivity

IEEE MILCOM